

OOPs Concepts

- Simula is considered the first object-oriented programming language. The programming paradigm where everything is represented as an object is known as a truly object-oriented programming language.
- Smalltalk is considered the first truly object-oriented programming language.
- The popular object-oriented languages are Java, C#, PHP, Python, C++, etc.
- Javascript is object based language where inheritance is not supported.

OOPs Concepts



OOPs Concepts

- Object

- Any entity that has state and behavior is known as an object. For example, a chair, pen, table, keyboard, bike, etc. It can be physical or logical.
- An Object can be defined as an instance of a class. An object contains an address and takes up some space in memory.
- Example: A dog is an object because it has states like color, name, breed, etc. as well as behaviors like wagging the tail, barking, eating, etc.

OOPs Concepts

Class

Collection of objects is called class. It is a logical entity.

A class can also be defined as a blueprint from which you can create an individual object.

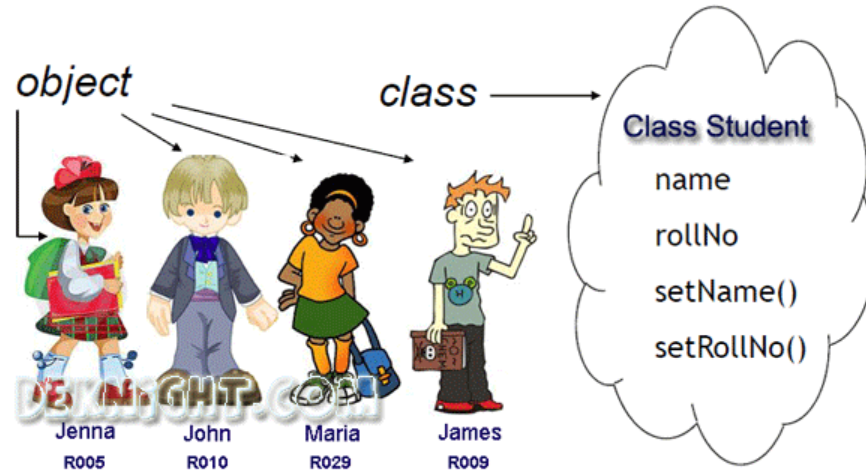
Class doesn't consume any space.

Class student

```
{  
    int rollno;  
    String name;  
    Void display();  
}
```

Struct student

```
{  
    int rollno;  
    String name; }  
}
```



OOPs Concepts

Abstraction

Hiding internal details and showing functionality is known as abstraction. For example phone call, we don't know the internal processing.

OOPs Concepts

Encapsulation

Binding (or wrapping) code and data together into a single unit are known as encapsulation. For example, a capsule, it is wrapped with different medicines.

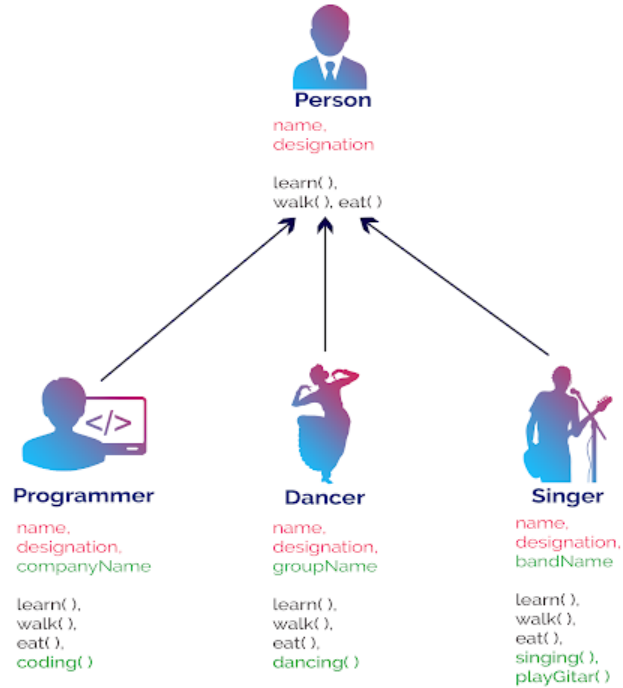
A java class is the example of encapsulation. Java bean is the fully encapsulated class because all the data members are private here.



OOPs Concepts

Inheritance

When one object acquires all the properties and behaviors of a parent object, it is known as inheritance. It provides code reusability. It is used to achieve runtime polymorphism.



OOPs Concepts

Polymorphism

If *one task is performed in different ways*, it is known as polymorphism. For example: to convince the customer differently, to draw something, for example, shape, triangle, rectangle, etc.

In Java, we **use method overloading and method overriding** to achieve polymorphism. Another example can be to speak something; for example, a cat speaks meow, dog barks woof, etc.

Comparison of procedure oriented programming and object oriented programming

OOPs makes development and maintenance easier, whereas, in a procedure-oriented programming language, it is not easy to manage if code grows as project size increases

OOPs provides data hiding, whereas, in a procedure-oriented programming language, global data can be accessed from anywhere.

OOPs provides the ability to simulate real-world event much more effectively. We can provide the solution of real word problem if we are using the Object-Oriented Programming language.

BASICS OF JAVA

- **What is JAVA ?**
- **Where JAVA is used ?**
- **What type of applications are created using JAVA ?**
- **Why use JAVA ?**

What is JAVA ?

JAVA is a **programming language** and a **platform**

What is Programming Language ?

- A programming language is a formal computer language designed to communicate instructions to a machine, particularly a computer.
- Programming languages can be used to create programs to control the behaviour of a machine or to express algorithms.

What is Programming Language ?

- The most basic (called low level) computer language is the machine language that uses binary(1's and 0's) code which is computer can run very fast without using translator (or) interpreter. But it is tedious and complex.

What is Programming Language ?

- The high - level languages (such as C, java) are much simpler to use but need to use another program to convert the high - level code into the machine code, therefore slower.

Example

Program to ADD 1 and 2

Assembly Language

- Store **1** at memory location say A
- Store **2** at memory location say B
- **ADD** contents of location A and B
- Store **RESULT**

Store 1 at memory location say A
Store 2 at memory location say B
ADD contents of location A and B
Store RESULT



Assembler

10100010



Store 1 at memory location say A
Store 2 at memory location say B
ADD contents of location A and B
Store RESULT



Assembler

10100010



Compiler



C - Language
`printf(1+2)`

C - Language

`printf(1+2)`



Compiler



.exe

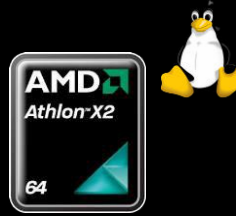


C - Language

`printf(1+2)`

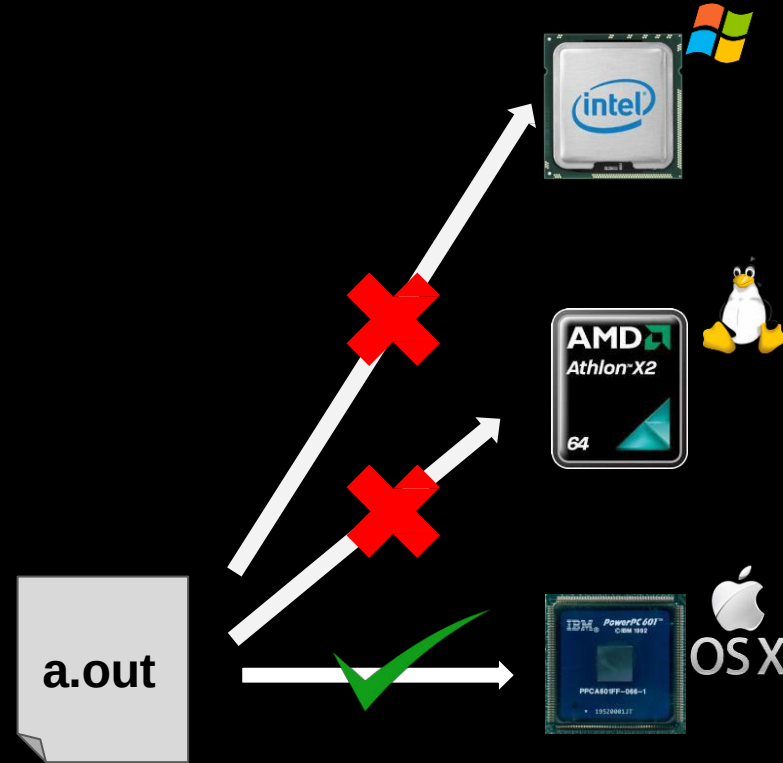
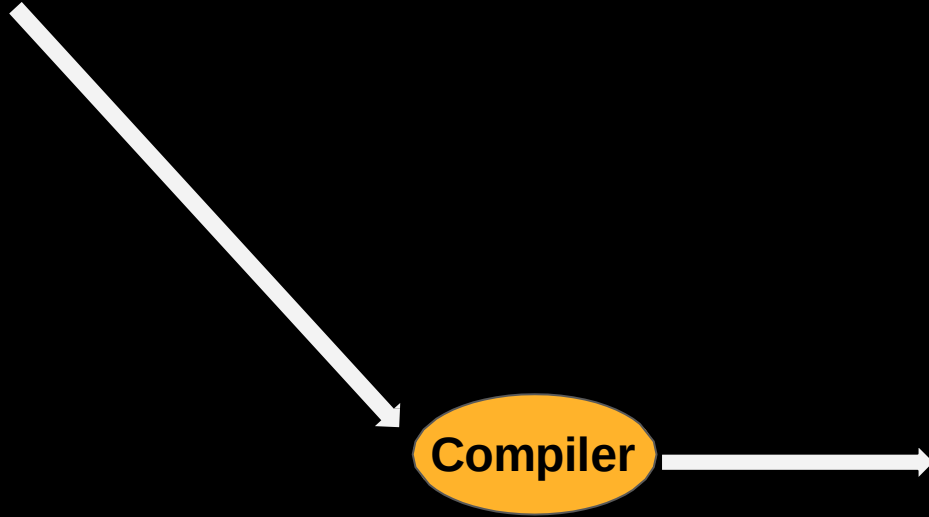
Compiler

a.out

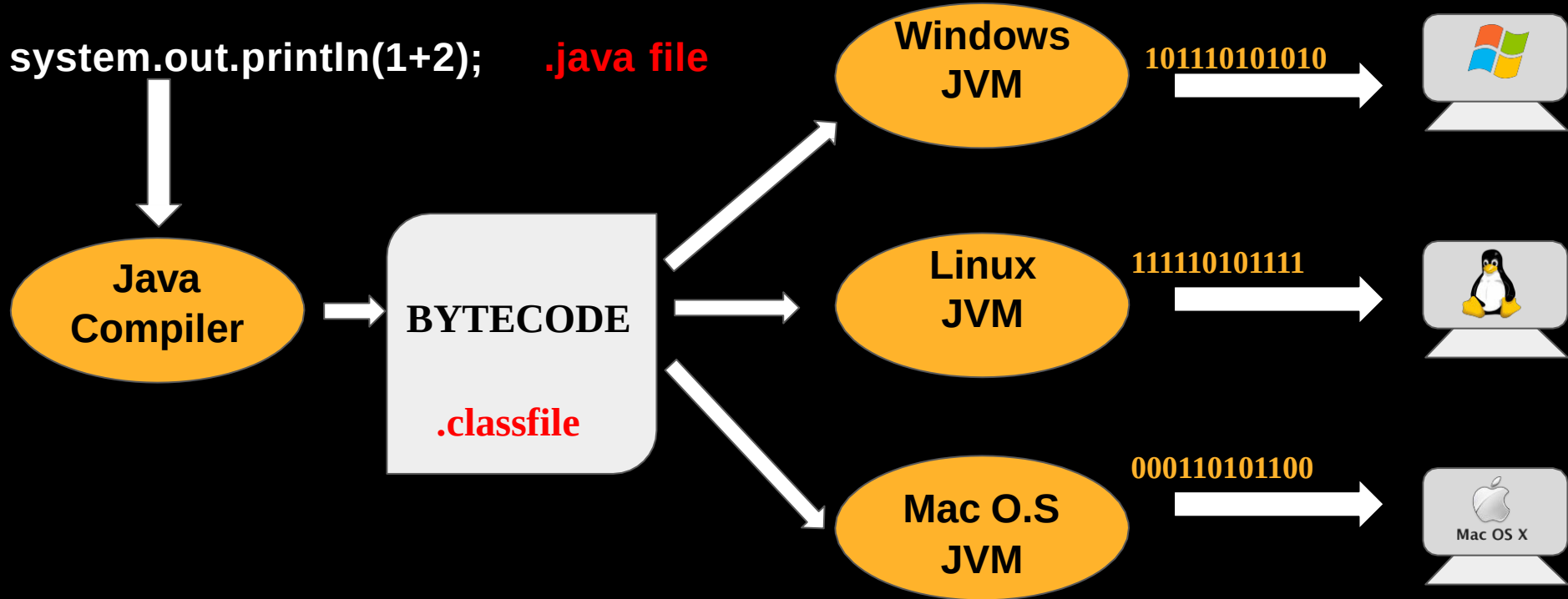


C - Language

`printf(1+2)`



Java - Language



History of Java

- The history of java starts from **Green Team**. Java Team members (also known as **Green Team**). Initiated a revolutionary task to develop a language for digital devices such as set-top boxes, televisions etc.
- **James Gosling, Mike Sheridan, and Patrick Naughton** initiated the Java language project in June 1991. The small team of sun engineers called **Green Team**.

History of Java

- Firstly, it was called “**Greentalk**” by James Gosling and file extension was **.gt**.
- After that, it was called **Oak** and was developed as part of the Green Project
- Why **Oak**?
Name is inspired by an **Oak** tree that stood outside of Gosling's office. **Oak** tree is a symbol of strength and chosen as a national tree of many countries like U.S.A, France, Germany etc.

History of Java

But in 1995, team had decided to change the name of language, because Oak name was already a trademark by Oak technologies.

New Name was Java

The suggested words were “dynamic”, “revolutionary”, “silk”, “joit”, “DNA” etc

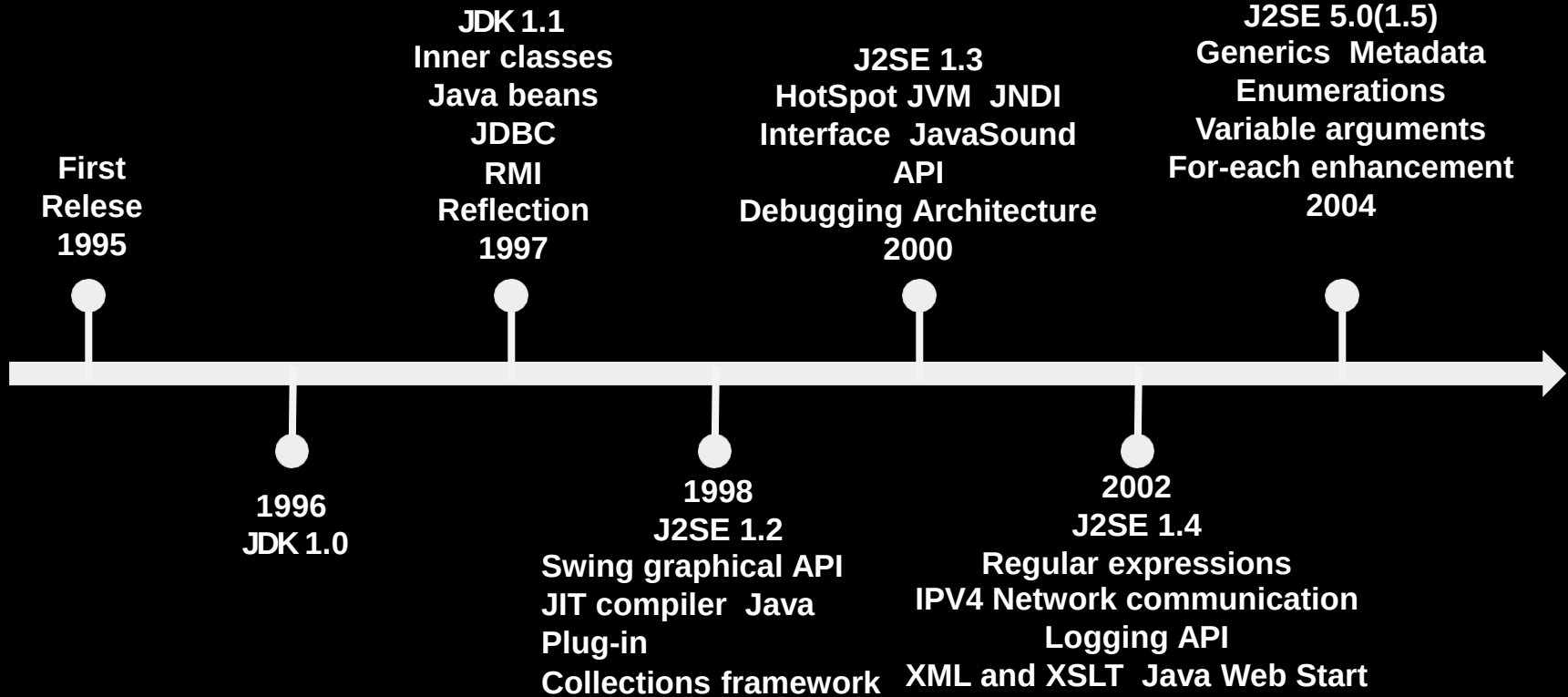
According to James Gosling “Java and Silk” were two top choices. Since java was so unique, most of the team members preferred java

History of Java

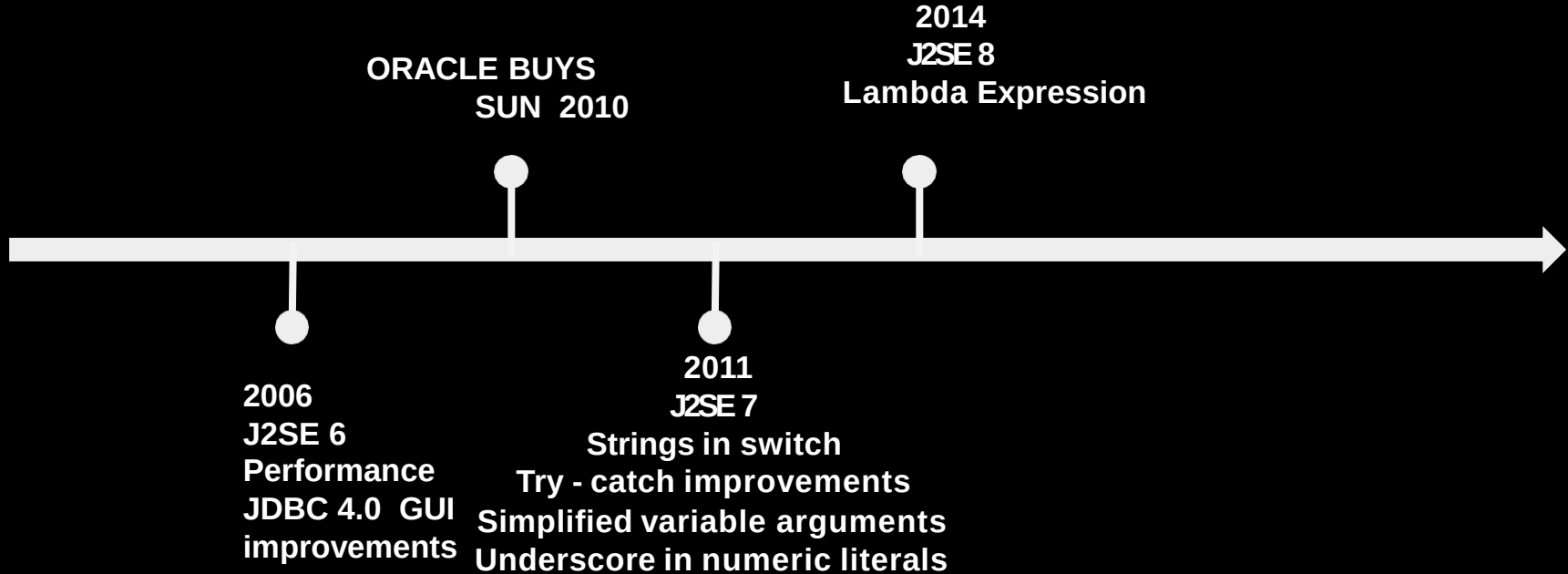
- **Java:** Java is an island of Indonesia where first coffee was produced (called java coffee)
- Java coffee consumed in large quantities by language creators.
- Java is just a name not an acronym.



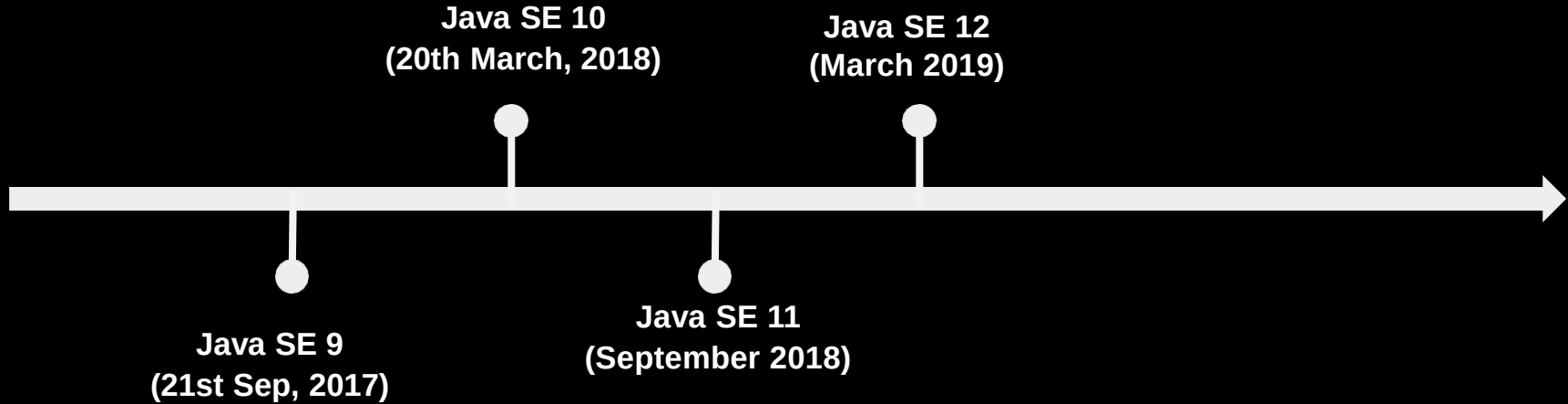
History of Java



History of Java

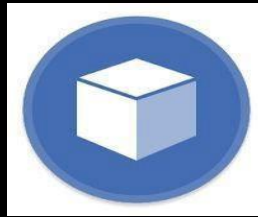


History of Java



Where java is used?

- According to Sun Microsystem, 3 billion devices run java. There are many devices where java is currently used. Some of them are..
- Desktop Applications:



Where java is used?

➤ Web Applications:



➤ Games:



Types of Java Applications

- **Standalone application:** It is also known as desktop application (or) window-based application.
- Standalone software is a computer software that can work offline i.e it doesn't necessarily require network connection.

Types of Java Applications

- **Web application:** An application that is stored on a remote server and delivered over the internet through a browser interface.
- **Enterprise application:** Enterprise software is computer software used to satisfy the needs of an organization rather than individual users. Such organizations would include business, schools, clubs, charities or governments.

Types of Java Applications

- **Mobile Application:** A mobile application, most commonly referred to as an app, is a type of application software designed to run on a mobile device, such as smartphone or tablet computer. Mobile applications frequently serve to provide users with similar services to those accessed on PCs. Apps are generally small, individual software units with limited function.

Features of Java or Java Buzzwords

- Simple
- Object - Oriented
- Platform independent
- Secured
- Robust
- Architecture neutral
- Portable
- Dynamic
- Interpreted
- High Performance
- Multi Threaded
- Distributed

Features of Java

- **Simple**- Java is very easy to learn and its syntax is simple, clean and easy to understand. According to Sun, Java language is a simple programming language because:
 - Java syntax is based on C++ (so easier for programmers to learn it after C++).
 - Java has removed many confusing and rarely-used features e.g. explicit pointers, operator overloading etc.
 - There is no need to remove unreferenced objects because there is Automatic Garbage Collection in java.

```
class twowheeler{
```

```
    Char color;
```

```
    speed();}ob;
```

Features of Java

➤ **Object Oriented :**

Basic concepts of Object Oriented programming are

1. **Object**
2. **Class**
3. **Inheritance**
4. **Polymorphism**
5. **Abstraction**
6. **Encapsulation**

Features of Java

➤ Platform Independent :

A platform is the hardware or software environment in which a program runs.

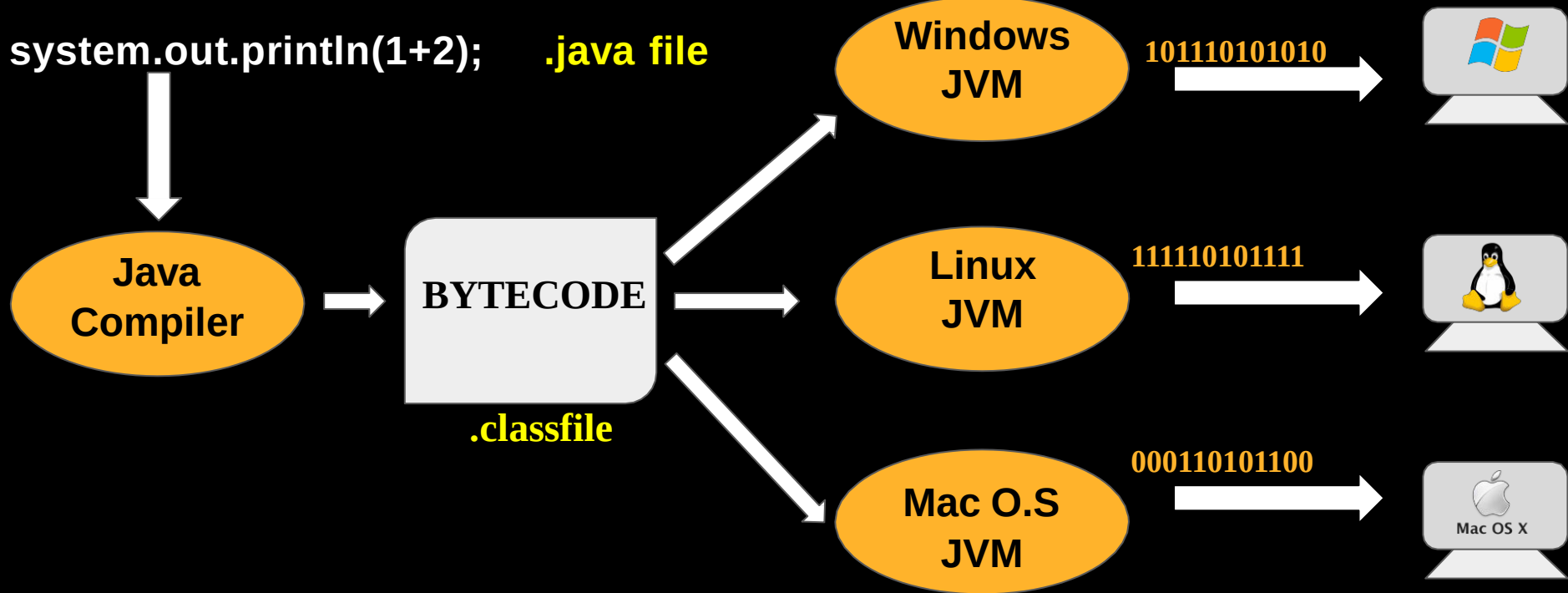
Features of Java

Java code can be run on multiple platforms e.g. Windows, Linux, Mac/OS etc.

Java code is compiled by the compiler and converted into bytecode.

This bytecode is a platform-independent code because it can be run on multiple platforms i.e. Write Once and Run Anywhere(WORA).

Features of Java



Features of Java

- **Secure:** It does not use explicit pointer. So, Java does not allow access of memory area directly which makes it more secure and keeps it safe from many sources of bugs.
- Java provides strong Exception Handling mechanism which makes it more secure as it does not allow to run the program until you rectify the error. Java provides access modifier/access specifier like private, protected, public and default. It helps in controlling the access of any member.

Features of Java

- **Secure:**
- **Java Programs run inside a virtual machine sandbox**
- **Bytecode Verifier:** It checks the code fragments for illegal code that can violate access rights to objects.

Features of Java

- **Robust:** Two main problems that cause program failures are memory management mistakes and mishandled runtime errors. Java handles both of them efficiently.
- **Memory management** mistakes can be overcome by garbage collection. Garbage collection is automatic deallocation of objects which are no longer needed. **Mishandled runtime errors** are resolved by Exception Handling procedures.

Features of Java

Portable:

- When we compile the java program, the compiler of java generates .class file that contains the Bytecodes of your java program.
- The generated Bytecodes are secure and can run on any machine (portable) which has JVM.

Features of Java

➤ Architecture - In neutral:

java there is no implementation dependent feature e.g. size of primitive types is fixed.

In C programming, float data type occupies 4 bytes of memory for 32-bit architecture and 8 bytes of memory for 64-bit architecture. But in java, it occupies 4 bytes of memory for both 32 and 64 bit architectures.

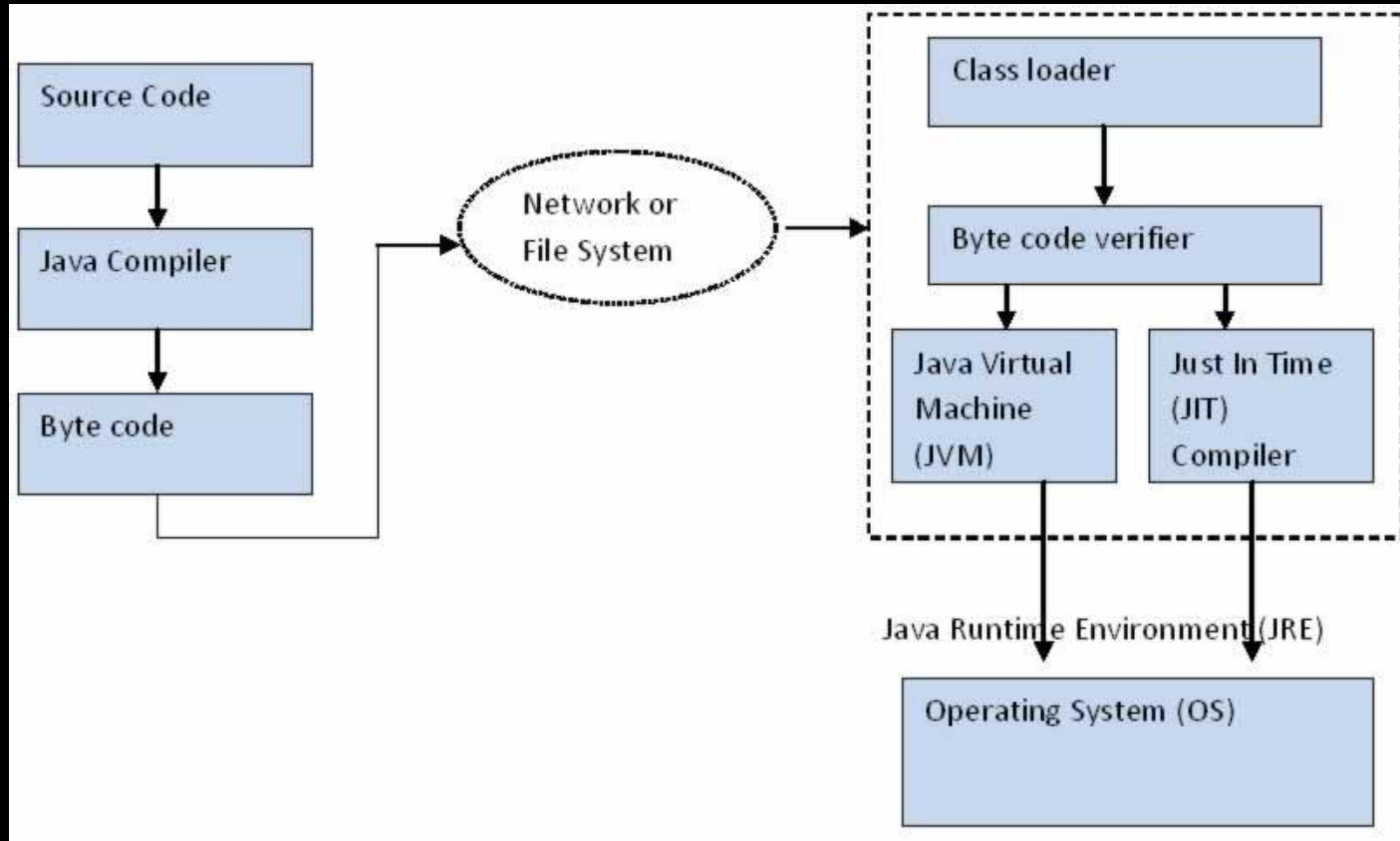
Features of Java

- **Multi threaded:** Java supports multithreading. It enables a program to perform several tasks simultaneously.
- **Interpreted and High Performance:** Java interpreter converts the bytecode into processor readable binary code. For very high performance it uses a Just-in-time compiler.

Features of Java

- **Distributed:** RMI(Remote Method Invocation) and EJB(Enterprise Javabeans) are used for creating distributed applications in java.
- In simple words: The java programs can be distributed on more than one systems that are connected to each other using internet connection. Objects on one JVM (java virtual machine) can execute procedures on a remote JVM.

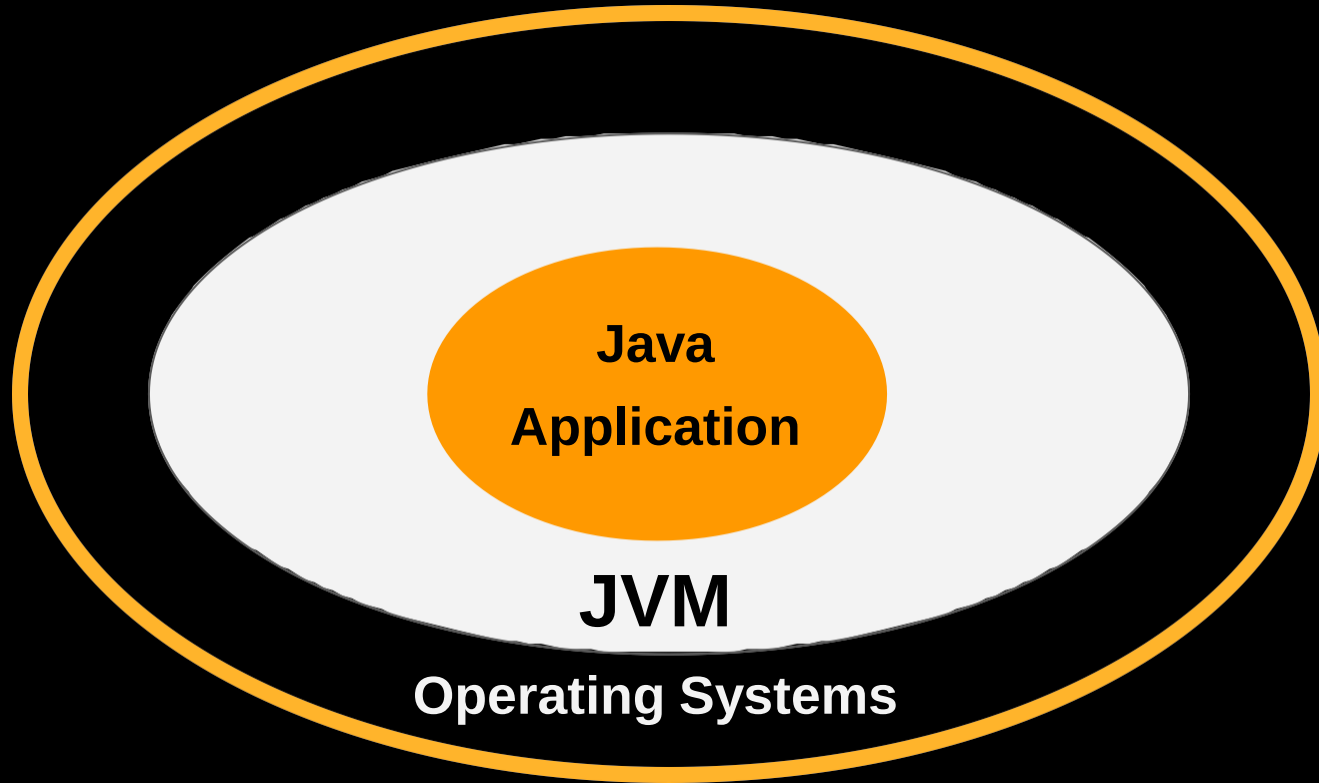
Difference Between JDK, JVM and JRE



Java Virtual Machine (JVM)

- JVM is the heart of Java programming Language.
- As we all aware when we compile a java file, output is not an “.exe” but it's a ‘.class’ file. ‘.class’ file consists of Java bytecode.
- Java Virtual Machine interprets the bytecode into the machine code depending upon the underlying operating system and Hardware combination.

Java Virtual Machine (JVM)



Lifetime of a Java Virtual Machine

- When a java application starts, a runtime instance of JVM is created. When the application completes, the instance terminates.
- If you start three java applications at the same time, on the same computer, you will get three java virtual machine instances running
- Each java application runs inside its own java virtual Machine.

**Application
1**

**Application
2**

**Application
3**

JVM

JVM

JVM

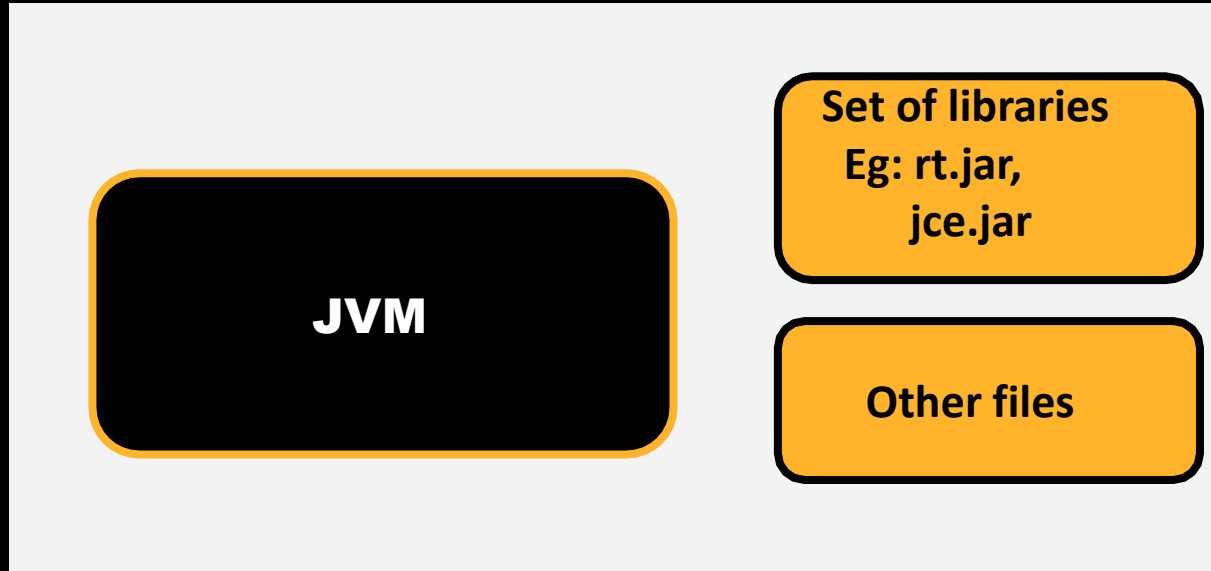
OS

Hardware

Java Runtime Environment

- **Java runtime environment contains JVM, class libraries and other supporting files.**
- **It doesn't contain any development tools such as compiler, debugger etc.**
- **Actually JVM runs the program, and it uses the class libraries, and other supporting files provided in Java runtime environment.**

Java Runtime Environment



JAVA RUNTIME ENVIRONMENT

JAVA DEVELOPMENT KIT

- **Java development kit provides all necessary tools to develop any java application as well as runs it.**
- **JDK = Tools required to develop java application
+
Java Runtime Environment**

JAVA DEVELOPMENT KIT

- **The tools provided by JDK to develop java application are listed below**
- **Java Compiler**
- **Java Debugger**
- **Java appletviewer etc.**

Java Development Kit



Compiler

Debugger

Java
applet
viewer

JDK

JUST-IN-TIME Compiler

- Just-in-time compiler is a part of JVM that is used to speed up the execution time of a program.
- Basic definition of compiler is “a program that translates the source code to a machine executable code”.

So, How Just-in-time compiler differs?

JUST-IN-TIME Compiler

- When compiling a java program, the static compiler that is run using the command **javac** converts the source code to bytecode which are in the form of .class files. These .class files (or) byte code needs to be interpreted to convert it into a machine executable code

JUST-IN-TIME Compiler

- Bytecode of a program may usually consists of methods, variables, threads and other instructions
- So, if there is a repeated code, instead of interpreting it every time the code is compiled to an executable code by the JIT compiler.
- JIT compiles the code when it is needed but not before runtime.

JUST-IN-TIME Compiler

- Whenever a program is executed this compiled code is invoked instead of interpreting the entire byte code and is quite efficient, and increases the performance of the program as well.
- However, the use of JIT is optional, but SUN microsystems suggest that it is quite efficient to use JIT especially if there exists a repeated code in the program.

Overview of How JIT compiler helps in compilation of code

- **Using a compiler, java source code is converted to byte code.**
- **JVM loads the .class files at runtime and converts them to a machine understandable code using an interpreter.**
- **JIT compiler is a feature of JVM which when enabled makes the JVM analyze the method calls in byte code and compiles them to more efficient code.**

Overview of How JIT compiler helps in compilation of code

- JIT optimizes the code.
- The JVM then executes this optimized code instead of interpreting it. Which is likely to increase the performance of the execution.

Creating Java Program

```
class FirstProgram {  
    public static void main(String args[]) {  
        System.out.println("First Java Program");  
    }  
}
```

Compiling a Java Program

- A compiler is an application that translates programs from the Java language to a language more suitable for executing on the computer.
- It takes a text file with the **.java** extension as input (your program) and produces a file with a **.class** extension (the computer-language version).

Compiling a Java Program

- To compile **FirstProgram.java** type the following command at the terminal.

```
javac FirstProgram.java
```

Executing a java program

- Once you compile your program, you will get .class file. Now, we can execute the .class file by following command

```
java FirstProgram
```

OUTPUT: First Java Program

Understanding First Java Program

- **class** keyword is used to declare a class in java.
- **public** keyword is an access modifier which represents visibility, it means it is visible to all.
- **void** is the return type of the method, it means it doesn't return any value.
- **main** represents startup of the program.

Understanding First Java Program

- **static** is a keyword, if we declare any method as static, it is known as static method.
 - The core advantage of static method is that there is no need to create object to invoke the static method.
 - The main method is executed by the JVM, so it doesn't require to create object to invoke the main method. so it saves memory.

Understanding First Java Program

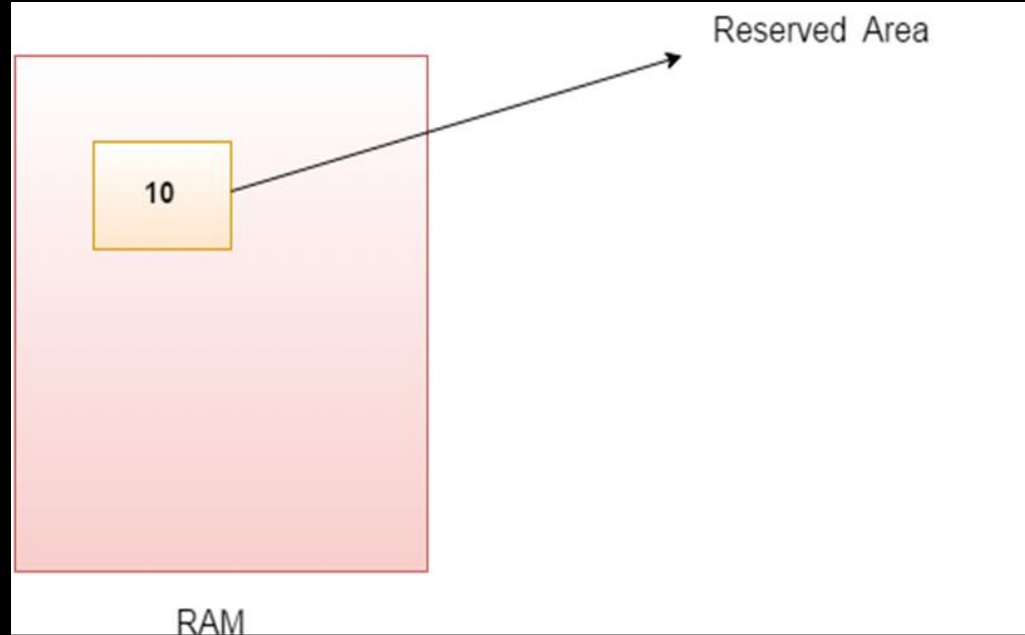
- **String[] args** is used for command line argument.
- **System.out.println()** is used print statement.

JAVA VARIABLES

Variables

- A variable is a container which holds the value while the java program is executed. A variable is assigned with a datatype.
- Variable is a name of memory location. There are three types of variables in java: local, instance and static.

VARIABLE IS NAME OF RESERVED AREA ALLOCATED IN MEMORY.



TYPES OF VARIABLES

- **local variable** - A variable declared inside the body of the method is called local variable.
- **instance variable** - A variable declared inside the class but outside the body of the method, is called instance variable. It is not declared as static.
- **static variable** - A variable which is declared as static is called static variable. It cannot be local. You can create a single copy of static variable and share among all the instances of the class.

EXAMPLE

class A

{

int data=50; **//instance variable**

static int m=100; **//static variable**

void method()

{

int n=90; **//local variable**

}

} **//end of class**

Class B

{

public.....{ A ob,ob1; ob.m=50; ob1.m=10;}}

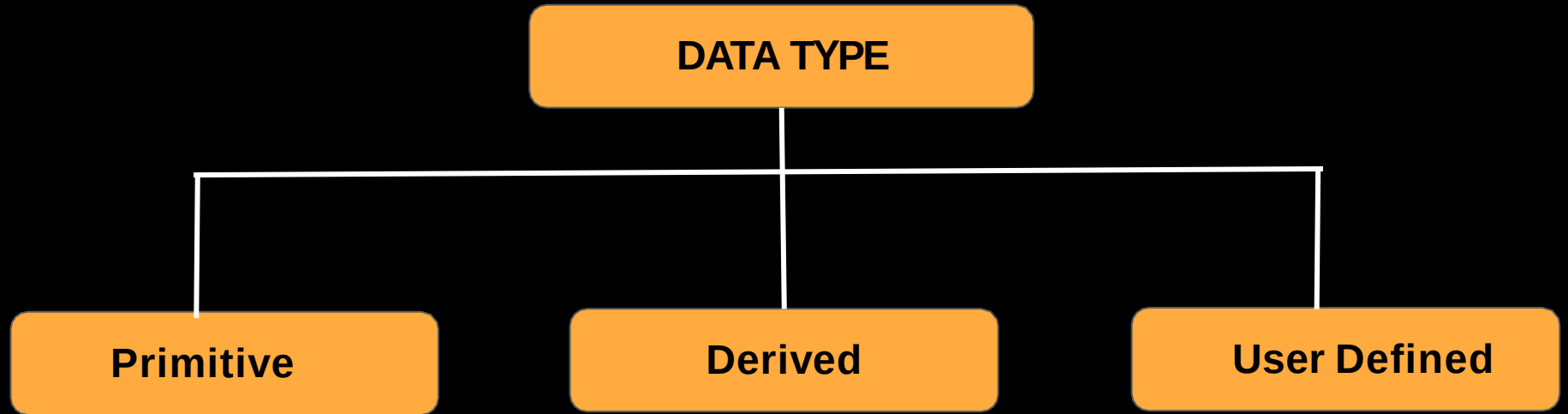
JAVA VARIABLE EXAMPLE: ADD TWO NUMBERS

```
class Simple{  
    public static void main(String[] args){  
        int a=10;  
        int b=10;  
        int c=a+b;  
        System.out.println(c);  
    }  
}
```

Data Types

- Data type is a special keyword used to allocate sufficient memory space for the data.
- In general every programming language is containing three categories of data types. They are
 1. Fundamental or primitive data types
 2. Derived data types
 3. User defined data types.

Data Types



Primitive Data Types

- Primitive data types are those whose variables allows us to store only one value but they never allows us to store multiple values of same type.

Example:

```
int a;
```

```
a = 10; // valid
```

```
a = 10, 20, 30; // not valid
```

Primitive Data Types

- There are eight built-in types supported by Java to support:
- Four Integer Type
- Two Float Type
- One Character type
- One Boolean type.

Integer Type

- The integer types in Java are byte, short, int, and long. These four types differ only in the number of bits and, therefore, in the range of numbers each type can represent.
- All integral types represent signed numbers. There is no unsigned keyword as there is in C and C++.

Type	Contains	Default	Size	Range
byte	Signed integer	0	1 byte(8 bits)	-128 to +127
short	Signed integer	0	2 byte(16 bits)	-32768 to +32767
int	Signed integer	0	4 byte(32 bits)	-2147483648 to 2147483647
long	Signed integer	0	8 byte(64 bits)	-9223372036854775808 to 9223372036854775807

Boolean Type

- The boolean type represents a truth value. There are only two possible values of this type, representing the two boolean states: on or off, yes or no, true or false.
- Java reserves the words **true** and **false** to represent these two boolean values.
- **Note:** boolean values can never be converted to or from other data types

Boolean Type

Type	Contains	Default	Size	Range
boolean	true or false	false	1 bit	NA

Float Type

- The float types in Java are float and double. These two types differ only in the number of bits and, therefore, in the range of numbers each type can represent.
- float is a 32-bit, single-precision floating-point value, and double is a 64-bit, double-precision floating-point value.

Float Type

Type	Contains	Default	Size	Range
float	IEEE 754 floating point	0.0	32 bits	$\pm 1.4\text{E-}45$ to $\pm 3.4028235\text{E}+38$
double	IEEE 754 floating point	0.0	64 bits	$\pm 4.9\text{E-}324$ to $\pm 1.7976931348623157\text{E}+308$

Char Type

- In Java, the data type used to store characters is **char**.
- Character is 2 byte(16 bits) wide in Java.
- Java uses **Unicode** to represent characters.
- Java support lot of Unicode symbols from many more human languages for this purpose, it requires 16 bits.
- The range of a char is 0 to 65,536.
- There are no negative chars.

Char Type

Type	Contains	Default	Size	Range
char	Unicode character	\u0000	16 bits	\u0000 to \uFFFF

WHY CHAR USES 2 BYTE IN JAVA AND WHAT IS \U0000 ?

It is because java uses Unicode system not ASCII code system. The \u0000 is the lowest range of Unicode system.

Before Unicode, there were many language standards:ASCII (American Standard Code for Information Interchange) for the United States.

ISO 8859-1 for Western European Language.

KOI-8 for Russian.

GB18030 and BIG-5 for chinese, and so on.

WHY CHAR USES 2 BYTE IN JAVA AND WHAT IS \U0000 ?

- This caused two problems: A particular code value corresponds to different letters in the various language standards.
- The encodings for languages with large character sets have variable length. Some common characters are encoded as single bytes, other require two or more byte.
- Solution
- To solve these problems, a new language standard was developed i.e. Unicode System. In unicode, character holds 2 byte, so java also uses 2 byte for characters. lowest value: \u0000 highest value: \uFFFF
- Java suitable for representing strings in any language supported by Unicode.
- Unicode standard uses hexadecimal to express a character.
- the char data type was originally used to represent a 16-bit Unicode code point

Derived Data Types

- Derived data types are those whose variables allow us to store multiple values of same type. But they never allows to store multiple values of different types.
- In general derived data type can be achieve using array.
- Example:

```
int a[ ] = {10,20,30}; // valid
```

```
int b[ ] = {100, 'A', "ABC"}; // invalid
```

User Defined Data types

- **User defined data types are those which are developed by programmers by making use of appropriate features of the language.**
- **User defined data types related variables allows us to store multiple values either of same type or different type or both.**
- **This is a data type whose variable can hold more than one value of dissimilar type, in java it is achieved using class concept.**

User Defined Data types

- **Note:** In java both **derived and user defined** data types combined name as **reference data type**.
- In C language, user defined data types can be developed by using struct, union, enum etc.
- In java programming user defined data type can be developed by using the features of classes and interfaces.

Type Casting

- The process of converting one data type to another is called casting.
- Casting is often necessary when a function returns a data in different form than what we need.
- Under certain circumstances Type conversion can be carried out automatically, in other cases it must be "forced" manually (explicitly)

Type Casting

- The storage size of the types while casting is very important otherwise the data loss will occur.
- For example, suppose we cast a long to an int. A long is a 64-bit value and an int is a 32-bit value. When casting a long to an int, the compiler truncates the upper 32 bits of the long value so as to fit into the 32-bit int.
- If the upper 32 bits of the long contain any useful information, then data loss will occur.

Type Casting

- In some cases, the data type of the expression is changed automatically to the variable's data type. For example, suppose that `i` is an integer variable declared as follows:

```
int i = 10 ;
```

- Even though `d` is a variable of type `double`, the following assignment is valid:

```
double d = i *10; // valid,  
(i*10) is converted to type double
```

Type Casting

- Java changes $(i*10)$ from **int** to **double**, and then assigns it to d.
- This conversion does not cause any loss of information because the double data type is 64 bits wide and thus is wider than the int data type that has 32 bits. This is called a **widening conversion** because a narrower type is converted to a wider type, and it takes place implicitly.

Type Casting

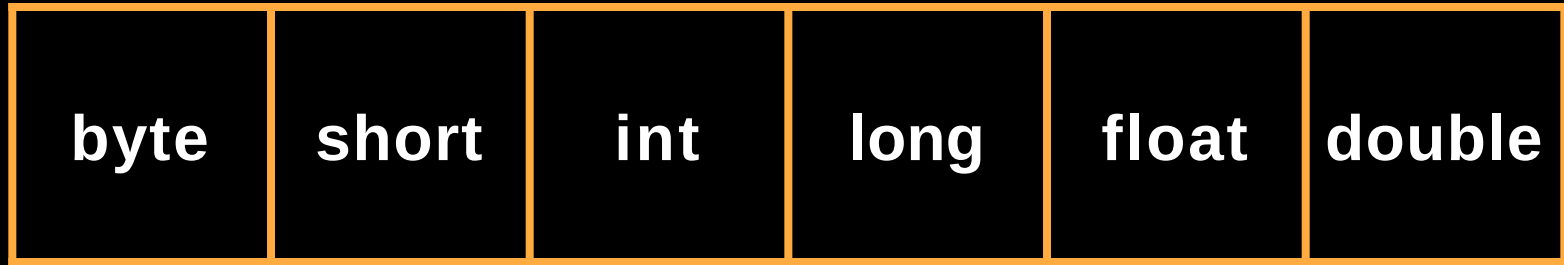
➤ some more examples of when widening conversions occur:

- `short s = 10;`
- `int num= s;`

- `float f = 10.5f;`
- `double d = f;`

Type Casting

- Following Figure summarizes the conversion rules: A conversion occurs from a given data type to any of the data types on its right.



Conversion to the wider type on the right takes place implicitly

Type Casting

- Assigning a value of type double to an int causes a compilation error:

```
double d = 100.5;
```

```
int i = d; // error!
```

- Java does not allow this assignment because a double can hold a larger range of values than an int, and it could result in a **loss of data**.

Type Casting

- To allow this type of assignment, it is necessary for the programmer to **force** a conversion from double to int using a **cast**.
- For example, a cast is used here to force a conversion of the variable d to type int and then assign it to i:

```
int i = (int) d; // i = 100
```
- Note: The above assignment truncates (not rounds) the fractional part so that it assigns 100 to i.

Overview of Type Casting

- **Automatic Conversion:** In Java type conversions are performed automatically when the type of the expression on the right hand side of an assignment operation can be safely promoted to the type of the variable on the left hand side of the assignment. Thus we can safely assign:

byte -> short -> int -> long -> float -> double.

Type Casting

- **Explicit Conversion (Casting):** we cannot automatically convert a long to an int because the first requires more storage than the second and consequently information may be lost.
- To force such a conversion we must carry out an explicit conversion (assuming that the long integer will fit into a standard integer). This is done using a process known as a **type cast**

Type Casting

```
myInteger = (int) myLongInteger
```

This tells the compiler that the type of myLongInteger must be temporarily changed to an int when the given assignment statement is processed. Thus, the cast only lasts for the duration of the assignment.

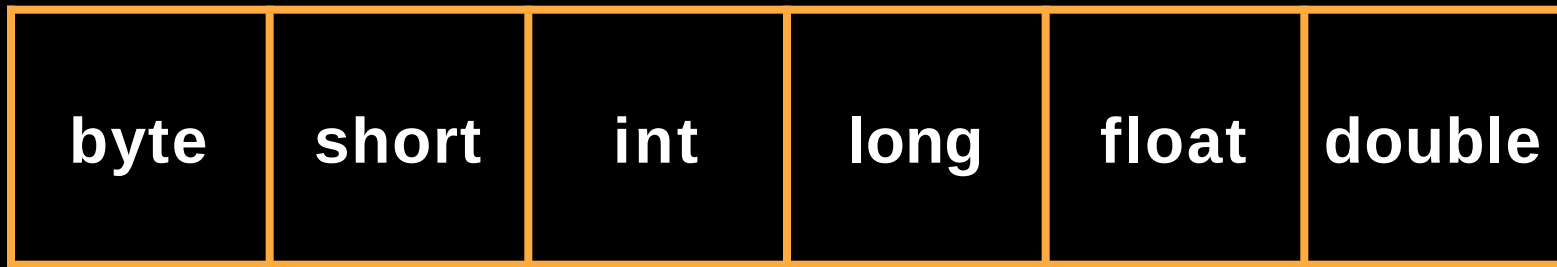
Type Casting

```
public class Main {  
    public static void main(String[ ] args) {  
        double myDouble = 9.78;  
        int myInt = (int) myDouble; // Manual casting: double to int  
        System.out.println(myDouble);  
        System.out.println(myInt);  
    }  
}
```

Type Casting

```
public class Main {  
    public static void main(String[ ] args) {  
        double myDouble = 9.78;  
        int myInt = (int) myDouble; // Manual casting: double to int  
        System.out.println(myDouble); // Outputs 9.78  
        System.out.println(myInt);    // Outputs 9  
    }  
}
```

Type Casting



Conversion to the narrower type on the right takes place explicitly

JAVA VARIABLE EXAMPLE: WIDENING

```
class Simple{  
    public static void main(String[] args){  
        int a=10;  
        float f=a;  
        System.out.println(a);  
        System.out.println(f);  
    }  
}
```

Output:

10

10.0

JAVA VARIABLE EXAMPLE: NARROWING (TYPECASTING)

```
class Simple{  
    public static void main(String[] args){  
        float f=10.5f;  
        //int a=f;//Compile time error  
        int a=(int)f;  
        System.out.println(f);  
        System.out.println(a);  
    }  
}
```

Output:

10.5

10

HOW TO GET INPUT FROM USER IN JAVA

Java Scanner Class

There are various ways to read input from the keyboard, the **java.util.Scanner** class is one of them.

The **Java Scanner** class breaks the input into tokens using a delimiter that is whitespace by default. It provides many methods to read and parse various primitive values.

Syntax

```
Scanner sc=new Scanner(System.in);
```

The above statement creates a constructor of the Scanner class having System.in as an argument. It means it is going to read from the standard input stream of the program. The java.util package should be import while using Scanner class.

Methods of Java Scanner Class

Java Scanner class provides the following methods to read different primitives types:

Method	Description
<code>int nextInt()</code>	It is used to scan the next token of the input as an integer.
<code>float nextFloat()</code>	It is used to scan the next token of the input as a float.
<code>double nextDouble()</code>	It is used to scan the next token of the input as a double.
<code>byte nextByte()</code>	It is used to scan the next token of the input as a byte.
<code>String nextLine()</code>	Advances this scanner past the current line.
<code>boolean nextBoolean()</code>	It is used to scan the next token of the input into a boolean value.
<code>long nextLong()</code>	It is used to scan the next token of the input as a long.
<code>short nextShort()</code>	It is used to scan the next token of the input as a Short.
<code>BigInteger nextBigInteger()</code>	It is used to scan the next token of the input as a BigInteger.
<code>BigDecimal nextBigDecimal()</code>	It is used to scan the next token of the input as a BigDecimal.

Integer input from user

```
import java.io.*;
import java.util.Scanner;
class ScannerTest{
    public static void main(String args[])
    {
        Scanner sc=new Scanner(System.in);
        System.out.println("Enter your rollno");
        int rollno= sc.nextInt();
        System.out.println("Enter your name");
        String name= sc.next();
        System.out.println("Enter your fee");
        double fee= sc.nextDouble();
        System.out.println("Rollno:"+rollno+" name:"+name+" fee:"+fee);
        sc.close();
    } }
```

Output

Enter your rollno 111

Enter your name Ratan

Enter the fee 450000

Rollno:111 name: Ratan fee:450000

Objectives

1. Size of an int in Java is

16 bit

32 bit

64 bit

Depends on execution environment

Solution

B. 32 bit

2. Size of float and double in java is

32 and 64

64 and 64

32 and 32

64 and 32

32 and 64

3. In Java, the word true is _____

- A Java Keyword
- A boolean literal
- Same as Value 1
- Same as Value 0

3. Solution

A Boolean literal

4. Which of the following is not a valid flow control statement?

- a) exit()
- b) break
- c) continue
- d) return

Answer: a

Explanation: `exit()` is not a flow control statement in Java. `exit()` terminates the currently running JVM.

WHAT WILL BE THE OUTPUT OF THE PROGRAM?

```
class Test

{ public static void main(String [] args)

{ Test p = new Test();

  p.start(); }

void start()

{ boolean b1 = false;

  boolean b2 = fix(b1);

  System.out.println(b1 + " " + b2);

}

boolean fix(boolean b1)

{ b1 = true; return b1; } }
```

- A. true true
- B. false true
- C. true false
- D. false false

Solution

Answer: Option B

EVEN OR ODD

```
import java.util.*;
public class EvenOdd
{
    public static void main(String [] args)
    {
        int no;
        Scanner sc=new Scanner(System.in);
        System.out.print("\nEnter A Number:");
        no=sc.nextInt();
        if(no%2==1)
        {
            System.out.print("Odd Number");
        }
        else
        {
            System.out.print("Even No"); }}
    }
```

SUM OF TWO NUMBERS

```
import java.io.*;

class Addition

{ public static void main(String args[])throws Exception
{
int a,b,c;

BufferedReader br= new BufferedReader( new InputStreamReader(System.in));

System.out.println("Enter the two numbers to add:");

a=Integer.parseInt(br.readLine());

b=Integer.parseInt(br.readLine());

c = a+b;

System.out.println("\nSum of two numbers:"+ c);

}}
```


FACTORIAL OF A NUMBER

```
import java.util.Scanner;

class Factorial

{ public static void main(String args[])

{ int n, c, fact = 1;

System.out.println("Enter an integer to calculate it's factorial");

Scanner in = new Scanner(System.in);

    n = in.nextInt();

    if (n < 0)

System.out.println("Number should be non-negative.");

    else {

        for (c = 1; c <= n; c++)

            fact = fact*c;

System.out.println("Factorial of "+n+" is = "+fact); } } }
```

SQUARE OF A NUMBER

```
import java.util.Scanner;

public class SquareofNumber {

public static void main(String[] args)

{

int number, square;

Scanner sc = new Scanner(System.in);

System.out.print(" Please Enter any Number : ");

number = sc.nextInt();

square = number * number;

System.out.println("\n The Square of a Given Number " + number + " = " + square);

}

}
```

JAVA CONTROL STATEMENTS

Control Flow Statements

Decision Making statements

- **if statements**
- **switch statement**

Loop statements

- **do while loop**
- **while loop**
- **for loop**
- **for-each loop**

Jump statements

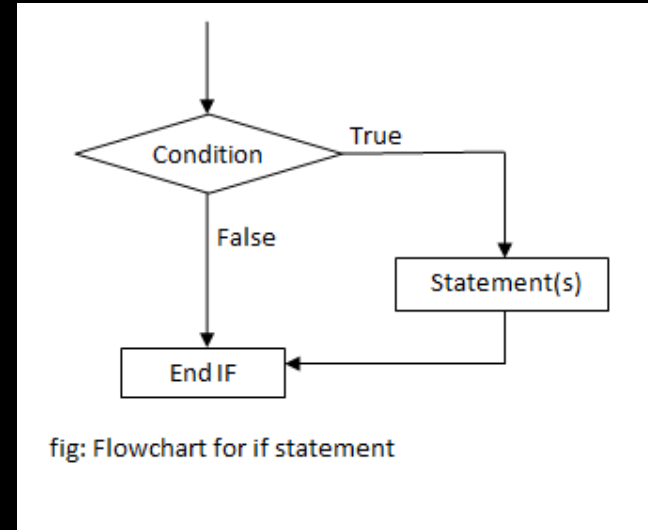
- **break statement**
- **continue statement**

If Statement

Use the if statement to specify a block of Java code to be executed if a condition is true.

Ex-1 `if (20 > 18) {`
 `System.out.println("20 is greater than 18");`
 `}`

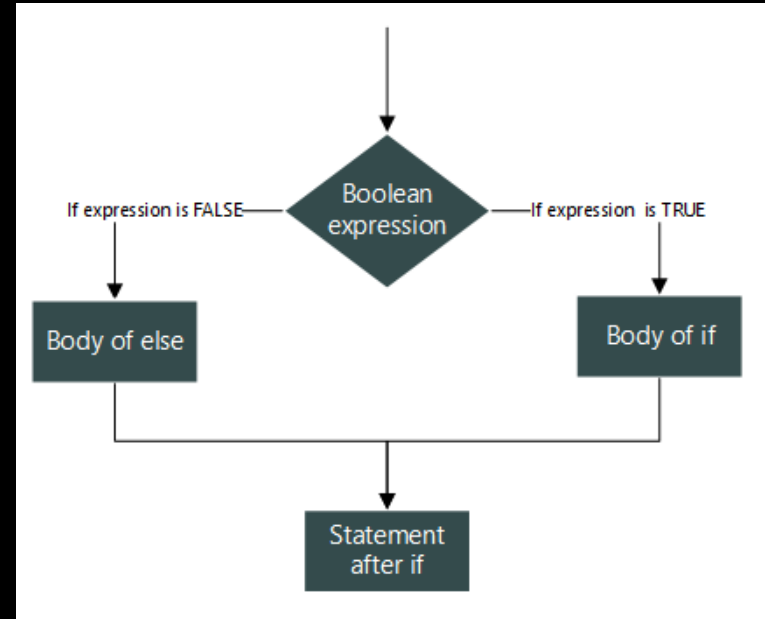
Ex-2 `int x = 20;`
 `int y = 18;`
 `if (x > y) {`
 `System.out.println("x is greater than y");`
 `}`



The if-else Statement

Use the **else** statement to specify a block of code to be executed if the condition is **false**.

```
int time = 20;  
if (time < 18) {  
    System.out.println("Good day.");  
} else {  
    System.out.println("Good evening.");  
}
```



The else if Statement

Use the **else if** statement to specify a new condition if the first condition is **false**.

```
int time = 22;  
if (time < 10) {  
    System.out.println("Good morning.");  
} else if (time < 20) {  
    System.out.println("Good day.");  
} else {  
    System.out.println("Good evening.");  
}
```

The else if Statement

Use the **else if** statement to specify a new condition if the first condition is **false**.

```
int time = 22;  
if (time < 10) {  
    System.out.println("Good morning.");  
} else if (time < 20) {  
    System.out.println("Good day.");  
} else {  
    System.out.println("Good evening.");  
}
```

Output:-Good evening.

Java Switch Statements

Use the **switch** statement to select one of many code blocks to be executed.

```
int day = 1;
switch (day) {
    case 1:
        System.out.println("Today is Saturday");
        break;
    case 2:
        System.out.println("Today is Sunday");
        break;
    default:
        System.out.println("Looking forward to the Weekend");
}
```

Java Switch Statements

Use the **switch** statement to select one of many code blocks to be executed.

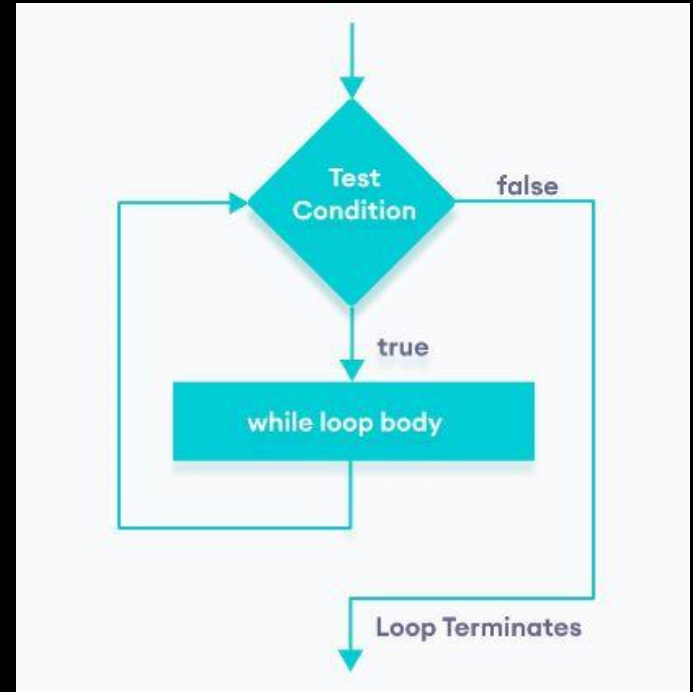
```
int day = 1;
switch (day) {
    case 1:
        System.out.println("Today is Saturday");
        break;
    case 2:
        System.out.println("Today is Sunday");
        break;
    default:
        System.out.println("Looking forward to the Weekend");
}
```

Output:-Today is Saturday

The While Loop

The **while** loop loops through a block of code as long as a specified condition is true.

```
int i = 0;  
while (i < 5) {  
    System.out.println(i);  
    i++;  
}
```

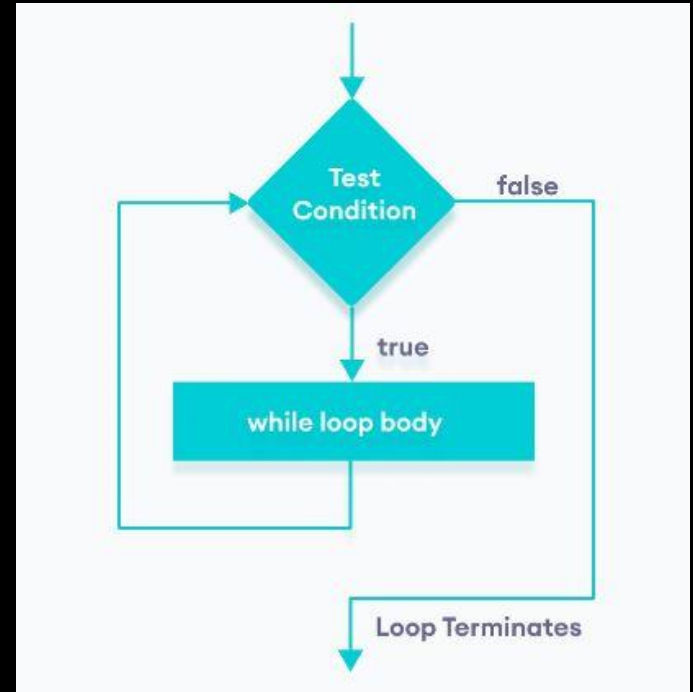


The While Loop

The **while** loop loops through a block of code as long as a specified condition is true.

```
int i = 0;  
while (i < 5) {  
    System.out.println(i);  
    i++;  
}
```

Output:- 0 1 2 3 4

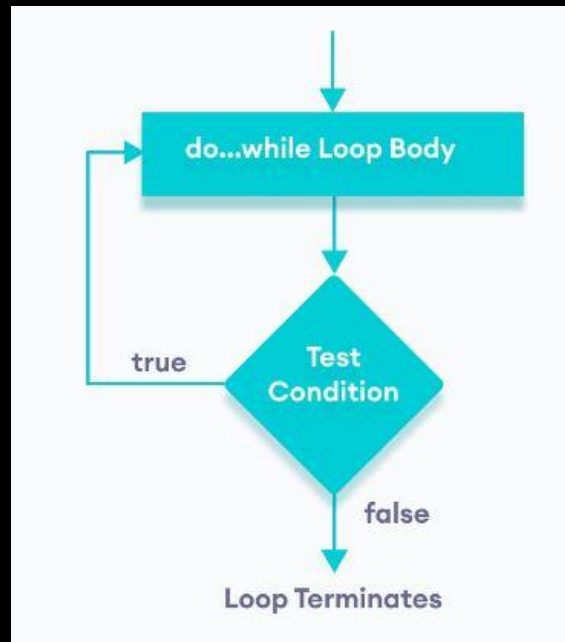


The Do/While Loop

The **do/while** loop is a variant of the **while** loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

```
int i = 0;  
do {  
    System.out.println(i);  
    i++;  
}  
while (i < 5);
```

Output:- 0 1 2 3 4

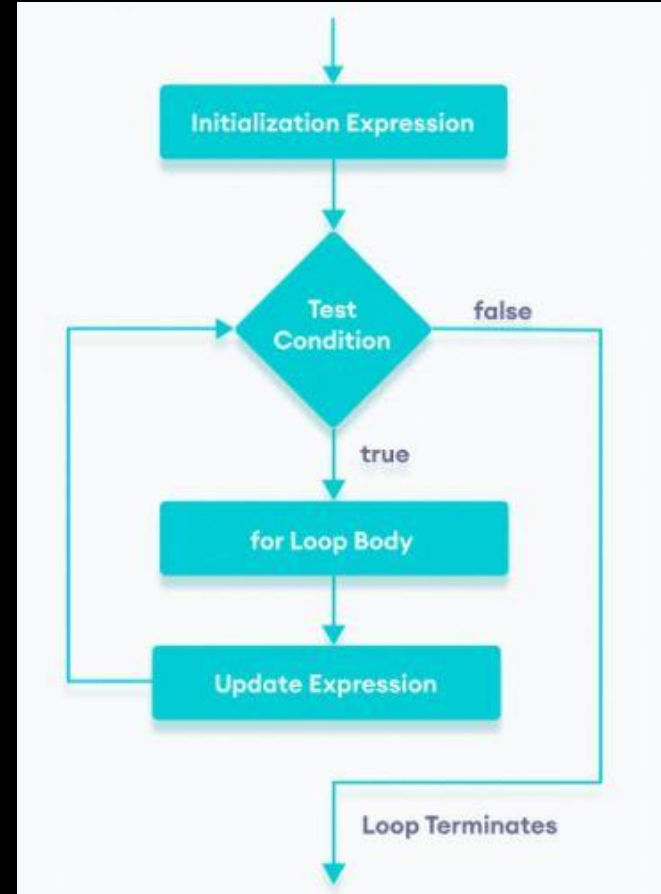


Java For Loop

```
for (int i = 0; i < 5; i++)  
{  
    System.out.println(i);  
}
```

```
for (int i = 0; i <= 10; i = i + 2)  
{  
    System.out.println(i);  
}
```

Output:- 0 2 4 6 8 10



Exercise

Example 3.1 Using Nested if Statements

This program reads in number of years and loan amount and computes the monthly payment and total payment. The interest rate is determined by number of years.

ARRAY

ARRAYS IN JAVA

An array is a group of like-typed variables that are referred to by a common name. Arrays in Java work differently than they do in C/C++. Following are some important point about Java arrays.

- In Java all arrays are **dynamically allocated**.
- Since **arrays are objects in Java**, we can find their length using member length. This is different from C/C++ where we find length using sizeof.
- A Java array variable can also be declared like other variables with [] after the data type.
- The variables in the array are ordered and each have an index beginning from 0.
- Java array can be also be used as a static field, a local variable or a method parameter.

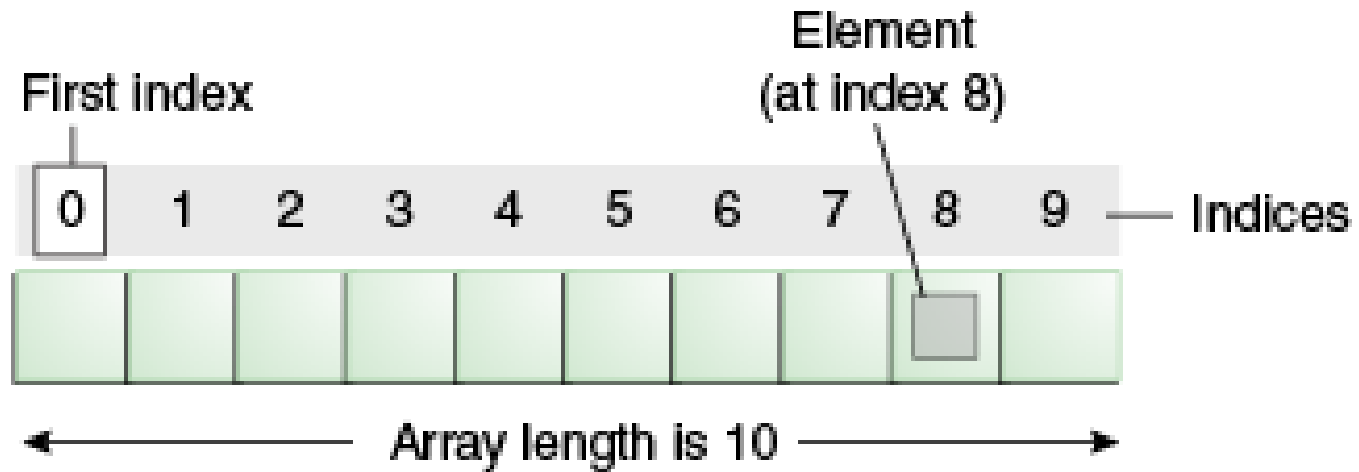
source: <https://www.javatpoint.com/java-tutorial>

- The size of an array must be specified by an int value and not long or short

Java array is an object that contains elements of similar data type. It is a data structure where we store similar elements. We can store only a fixed set of elements in a Java array.

An array in Java is an object.

- In Java, there is a class for every array type, so there's a class for `int[]` and similarly for `float`, `double` etc.
- The direct superclass of an array type is `Object`. Every array type implements the interfaces `Cloneable` and `java.io.Serializable`.
- The direct super class of these classes are `Object.class`



Advantage of Java Array

- **Code Optimization:** It makes the code optimized, we can retrieve or sort the data easily.
- **Random access:** We can get any data located at any index position.

Disadvantage of Java Array

- **Size Limit:** We can store only fixed size of elements in the array. It doesn't grow its size at runtime. To solve this problem, collection framework is used in java.

TYPES OF ARRAY IN JAVA

There are two types of array.

- Single Dimensional Array
- Multidimensional Array

SINGLE DIMENSIONAL ARRAY IN JAVA

`dataType[] arr; (or)`

`dataType []arr; (or)`

`dataType arr[];`

INSTANTIATION OF AN ARRAY IN JAVA

`datatype varname[]=new datatype[values];`

`Classname objname=new classname(values);`

`int vararray[] =new int[size];`

EXAMPLE OF SINGLE DIMENSIONAL JAVA ARRAY

```
class Testarray{  
public static void main(String args[]){  
int a[]=new int[5];//declaration and instantiation  
a[0]=10;//initialization  
a[1]=20;  
a[2]=70;  
a[3]=40;  
a[4]=50;  
    //printing array  
for(int i=0;i<a.length;i++)//length is the property of array  
System.out.println(a[i]);  
}
```

source: <https://www.javatpoint.com/java-tutorial>

DECLARATION, INSTANTIATION AND INITIALIZATION OF JAVA ARRAY

```
int a[]={33,3,4,5};//declaration, instantiation and initialization
```

```
class Testarray1{  
public static void main(String args[]){  
  
    int a[]={33,3,4,5};//declaration, instantiation and initialization  
  
    //printing array  
    for(int i=0;i<a.length;i++)//length is the property of array  
        System.out.println(a[i]);  
  
    }}
```

MULTIDIMENSIONAL ARRAY IN JAVA

Syntax to Declare Multidimensional Array in java

- `dataType[][] arrayRefVar; (or)`
- `dataType [][]arrayRefVar; (or)`
- `dataType arrayRefVar[][]; (or)`
- `dataType []arrayRefVar[];`

EXAMPLE TO INSTANTIATE MULTIDIMENSIONAL ARRAY IN JAVA

- `int[][] arr=new int[3][3];`//3 row and 3 column

EXAMPLE TO INITIALIZE MULTIDIMENSIONAL ARRAY IN JAVA

- `arr[0][0]=1;`
- `arr[0][1]=2;`
- `arr[0][2]=3;`
- `arr[1][0]=4;`
- `arr[1][1]=5;`
- `arr[1][2]=6;`
- `arr[2][0]=7;`
- `arr[2][1]=8;`
- `arr[2][2]=9;`

EXAMPLE OF MULTIDIMENSIONAL JAVA ARRAY

```
class Testarray3{  
    public static void main(String args[]){  
        //declaring and initializing 2D array  
        int arr[][]={{1,2,3},{2,4,5},{4,4,5}};  
        //printing 2D array  
        for(int i=0;i<3;i++){  
            for(int j=0;j<3;j++){  
                System.out.print(arr[i][j]+" ");  
            }  
            System.out.println();  
        }  
    }  
}
```

source: <https://www.javatpoint.com/java-tutorial>

ADDITION OF 2 MATRICES IN JAVA

```
class Testarray5{  
    public static void main(String args[]){  
        //creating two matrices  
        int a[][]={{1,3,4},{3,4,5}};  
        int b[][]={{1,3,4},{3,4,5}};  
        //creating another matrix to store the sum of two matrices  
        int c[][]=new int[2][3];  
        //adding and printing addition of 2 matrices  
        for(int i=0;i<2;i++){  
            for(int j=0;j<3;j++){  
                c[i][j]=a[i][j]+b[i][j];  
                System.out.print(c[i][j]+" ");  
            }  
        }  
    }  
}
```

source: <https://www.javatpoint.com/java-tutorial>


```
int a[][]={{1,3,4}, -----[0,0] [0,1] [0,2]
           {3,4,5}}; ----- [1,0] [1,1] [1,2]
```

```
int b[][]=
    {{1,3,4},- -----[00] [0 1].....
    {3,4,5}};
```

//creating another matrix to store the sum of two matrices

```
int c[][]=new int[2][3];
```

//adding and printing addition of 2 matrices

```
for(int i=0;i<2;i++){
    for(int j=0;j<3;j++){
        c[i][j]=a[i][j]+b[i][j];
        System.out.print(c[i][j]+" ");
    } System.out.println();//new line
}
}
```

JAVA PROGRAM TO ACCEPT ARRAY ELEMENTS AND CALCULATE SUM

```
import java.util.Scanner;

public class Array_Sum

{public static void main(String[] args)

{

int n, sum = 0;

Scanner s = new Scanner(System.in);

System.out.print("Enter no. of elements you want in array:");

n = s.nextInt();

int a[] = new int[n];

System.out.println("Enter all the elements:");

for(int i = 0; i < n; i++)

{ a[i] = s.nextInt();

sum = sum + a[i];}

System.out.println("Sum:"+sum);

} }
```

STRINGS

In java, string is basically an object that represents sequence of char values.

An array of characters works same as java string.

For example:

```
String s="javatpoint";
```

```
System.out.println(s);
```

Java String class provides a lot of methods to perform operations on string such as `compare()`, `concat()`, `equals()`, `split()`, `length()`, `replace()`, `compareTo()`, `intern()`, `substring()` etc.

Use package **java.lang.String**

STRING ARRAY DECLARATION IN

JAVA

Just like any other type of array, we can use the square brackets to declare an array of String. There are two ways of doing this. The first one is to use the square bracket after the variable name, for example:

```
String myArray[];
```

The other way of declaring String Array in Java is to place the square brackets after the data type. For example:

```
String[] myArray;
```

Declaring and initializing String Array in Java with Size

```
String[] myArray= new String[3];
```

```
myArray[0] = "Cat";
```

```
myArray[1] = "Dog";
```

```
myArray[2] = "Elephant";
```

source: <https://www.javatpoint.com/java-tutorial>

STRING ARRAY LENGTH/SIZE

The property length of a String Array can be used to determine the number of elements in an Array. Here is an example usage:

```
String[] thisIsAStringArray = {"Apple", "Banana", "Orange"};
```

```
System.out.println( thisIsAStringArray.length );
```

ARRAY.LENGTH : LENGTH IS A FINAL VARIABLE APPLICABLE FOR ARRAYS. WITH THE HELP OF LENGTH VARIABLE, WE CAN OBTAIN THE SIZE OF THE ARRAY.
STRING.LENGTH() : LENGTH() METHOD IS A FINAL VARIABLE WHICH IS APPLICABLE FOR STRING OBJECTS. LENGTH() METHOD RETURNS THE NUMBER OF CHARACTERS PRESENTS IN THE STRING.

```
public class Test {  
  
    public static void main(String[] args)  
  
    {  
  
        // Here array is the array name of int type  
  
        int[] array = new int[4];  
  
        System.out.println("The size of the array is " + array.length);  
  
  
  
        // Here str is a string object  
  
        String str = "German";  
  
        System.out.println("The size of the String is " + str.length());  
  
    }  
}
```

source: <https://www.javatpoint.com/java-tutorial>

JAVA PROGRAM FOR LINEAR SEARCH

```
import java.util.Scanner;

class LinearSearch

{ public static void main(String args[])

{ int c, n, search, array[];

Scanner in = new Scanner(System.in);

System.out.println("Enter number of elements");

n = in.nextInt();

array = new int[n];

System.out.println("Enter " + n + " integers");

    for (c = 0; c < n; c++)

    {

array[c] = in.nextInt();

    }

}
```

source: <https://www.javatpoint.com/java-tutorial>

JAVA PROGRAM FOR LINEAR SEARCH

```
System.out.println("Enter value to find");

search = in.nextInt();

for (c = 0; c < n; c++)

{ if (array[c] == search) /* Searching element is present */

{ System.out.println(search + " is present at location " + (c + 1) + ".");

break; } }

if (c == n) /* Element to search isn't present */

System.out.println(search + " isn't present in array."); } }
```


OBJECTIVES

- I. In java arrays are
 - a. Object
 - b. Object reference
 - c. Primitive data type
 - d. None of the above

Answer

A

2. Which one of the following is the valid statements?

- a. `char [] c= new char [5];`
- b. `char [] c= new char ();`
- c. `char [] c= new char (4);`
- d. `char [] c= new char [];`

- Answer

A

Array Length

To find out how many elements an array has, use the length property:

```
String[ ] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
System.out.println(cars.length);
```

Output:- 4

For each loop

```
String[ ] cars = {"Volvo", "BMW", "Ford", "Mazda"};  
for (String i : cars)  
{  
    System.out.println(i);  
}
```

Output:- Volvo, BMW, Ford, Mazda

Java Multi-Dimensional Arrays

```
int[ ][ ] myNumbers = { {1, 2, 3, 4}, {5, 6, 7,8} };
```

```
int x = myNumbers[1][2];
```

```
System.out.println(x);
```

Output:- 7

OPERATORS

Operators

1. Arithmetic Operators.
2. The Bitwise Operators
3. Relational Operators
4. Boolean Logical Operators
5. Assignment Operators
6. The ? Operator
7. Operator Precedence

Arithmetic Operators

Operator	Name	Description	Example
+	Addition	Adds together two values	$x + y$
-	Subtraction	Subtracts one value from another	$x - y$
*	Multiplication	Multiplies two values	$x * y$
/	Division	Divides one value by another	x / y
%	Modulus	Returns the division remainder	$x \% y$
++	Increment	Increases the value of a variable by 1	<code>++x</code>
--	Decrement	Decreases the value of a variable by 1	<code>--x</code>

Arithmetic Operators

```
public class IncrementDecrementQuiz
{
    public static void main(String[ ] args)
    {
        int i = 11;

        i = i++ + ++i;

        System.out.println(i);
    }
}
```

Output:- 24

Arithmetic Operators

```
public class IncrementDecrementQuiz
{
    public static void main(String[ ] args)
    {
        int i = 11;

        int j = --(i++);

        System.out.println(j);
    }
}
```

Output:- Error

Relational Operators

Operator	Name	Example
==	Equal to	$x == y$
!=	Not equal	$x != y$
>	Greater than	$x > y$
<	Less than	$x < y$
>=	Greater than or equal to	$x >= y$
<=	Less than or equal to	$x <= y$

Boolean Logical Operators

Operator	Name	Description	Example
&&	Logical and	Returns true if both statements are true	<code>x < 5 && x < 10</code>
	Logical or	Returns true if one of the statements is true	<code>x < 5 x < 4</code>
!	Logical not	Reverse the result, returns false if the result is true	<code>!(x < 5 && x < 10)</code> <code>!(x < 5 x < 4)</code> <code>!(x < 5)</code>

Assignment Operators

Operator	Example	Same As
=	x = 5	x = 5
+=	x += 3	x = x + 3
-=	x -= 3	x = x - 3
*=	x *= 3	x = x * 3
/=	x /= 3	x = x / 3
%=	x %= 3	x = x % 3
^=	x ^= 3	x = x ^ 3

The ? Operator

Java ternary operator is the only conditional operator that takes three operands. It's a one-liner replacement for the if-then-else statement and is used a lot in Java programming

Syntax:

Variable = Expression1 ? Expression2: Expression3

```
if(Expression1)
{
    variable = Expression2;
}
else
{
    variable = Expression3;
}
```

The ? Operator

```
public class IncrementDecrementQuiz
{
    public static void main(String[ ] args)
    {
        int p=0;

        p = 5<2 ? 4 : 3;

        System.out.println(p);
    }
}
```

Output:- 3

The ? Operator

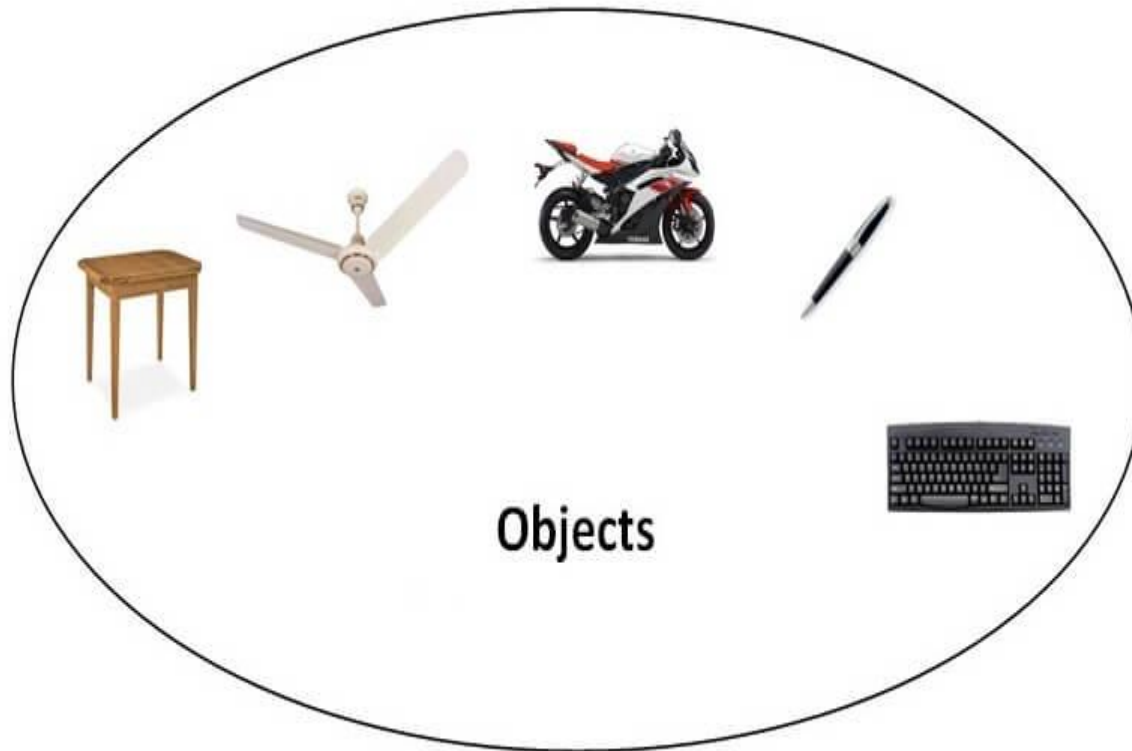
```
public class LargestNumberExample
{
    public static void main(String args[ ])
    {
        int x=69;
        int y=89;
        int z=79;
        int largestNumber = (x > y) ? (x > z ? x : z) : (y > z ? y : z);
        System.out.println("The largest numbers is: "+largestNumber);
    }
}
```

Output:- 89

OBJECT AND CLASS IN JAVA

Object is the physical as well as logical entity whereas class is the logical entity only.

OBJECT IN JAVA



An object has three characteristics:

- **state:** represents data (value) of an object.
- **behavior:** represents the behavior (functionality) of an object such as deposit, withdraw etc.
- **identity:** Object identity is typically implemented via a unique ID. The value of the ID is not visible to the external user. But, it is used internally by the JVM to identify each object uniquely.

OBJECT DEFINITIONS:

- Object is *a real world entity*.
- Object is *a run time entity*.
- Object is *an entity which has state and behavior*.
- Object is *an instance of a class*.

CLASS IN JAVA

A class is a group of objects which have common properties.

A class in Java can contain:

- **fields**
- **methods**
- **constructors**
- **blocks**
- **nested class and interface**

SYNTAX TO DECLARE A CLASS:

```
class <class_name>  
{  
    field;  
    method;  
}
```

METHOD IN JAVA

In java, a method is like function i.e. used to expose behavior of an object.

Advantage of Method

- Code Reusability
- Code Optimization

NEW KEYWORD IN JAVA

The new keyword is used to allocate memory at run time. All objects get memory in Heap memory area.

OBJECT AND CLASS EXAMPLE: MAIN WITHIN CLASS

```
class Student{  
    int id;//field or data member or instance variable  
    String name;  
    public static void main(String args[]){  
        Student s1=new Student();//creating an object of Student  
        System.out.println(s1.id);//accessing member through reference variable  
        System.out.println(s1.name);  
    }  
}
```

OBJECT AND CLASS EXAMPLE: MAIN OUTSIDE CLASS

```
class Student{  
    int id;  
    String name;  
}  
  
class TestStudentI{  
    public static void main(String args[]){  
        Student sI=new Student();  
        System.out.println(sI.id);  
        System.out.println(sI.name);  
    }  
}
```

3 WAYS TO INITIALIZE OBJECT

There are 3 ways to initialize object in java.

- By reference variable
- By method
- By constructor

Initializing object simply means storing data into object.

OBJECT AND CLASS EXAMPLE: INITIALIZATION THROUGH REFERENCE

```
class Student{  
    int id;  
    String name;  
}  
  
class TestStudent2{  
    public static void main(String args[]){  
        Student s1=new Student();  
        s1.id=101;  
        s1.name="Sonoo";  
        System.out.println(s1.id+" "+s1.name);//printing members with a white space  
    }  
}
```

INITIALIZATION THROUGH REFERENCE - CREATING MORE THAN ONE OBJECT

```
class Student{  
  
    int id;  
  
    String name;  
  
}  
  
class TestStudent3{  
  
    public static void main(String args[]){  
  
        //Creating objects  
  
        Student s1=new Student();  
  
        Student s2=new Student();  
  
        //Initializing objects  
  
        s1.id=101;  
  
        s1.name="Sonoo";  
  
        s2.id=102;  
  
        s2.name="Amit";  
  
        //Printing data  
  
        System.out.println(s1.id+" "+s1.name);  
  
        System.out.println(s2.id+" "+s2.name);  
  
    }  
  
}
```

source: <https://www.javatpoint.com/java-tutorial>

OBJECT AND CLASS EXAMPLE: INITIALIZATION THROUGH METHOD

```
class Student{  
    int rollno;  
    String name;  
    void insertRecord(int r, String n){  
        rollno=r;  
        name=n;  
    }  
    void displayInformation()  
    {System.out.println(rollno+" "+name);}  
}  
class TestStudent4{  
    public static void main(String args[]){  
        Student s1=new Student();  
        Student s2=new Student();  
        s1.insertRecord(111,"Karan");  
        s2.insertRecord(222,"Aryan");  
        s1.displayInformation();  
        s2.displayInformation();  
    }  
}
```

source: <https://www.javatpoint.com/java-tutorial>

AREA OF A RECTANGLE

```
class Rectangle{

    int length;

    int width;

    void insert(int l, int w){

        length=l;

        width=w;

    }

    void calculateArea(){System.out.println(length*width);}

}

class TestRectangle1{

    public static void main(String args[]){

        Rectangle r1=new Rectangle();

        Rectangle r2=new Rectangle();

        r1.insert(11,5);

        r2.insert(3,15);

        r1.calculateArea();

        r2.calculateArea();

    }

}
```

source: <https://www.javatpoint.com/java-tutorial>

JAVA PROGRAM TO FIND AREA AND PERIMETER OF A CIRCLE USING CLASS

```
import java.util.*;

class AreaOfCircle {

    float radius, area, perimeter;

    public void readRadius() {

        Scanner sc=new Scanner(System.in);

        System.out.print("Enter radius:");

        radius=sc.nextFloat(); }

    public float getArea()

    { area= Math.PI *radius*radius;

        return area; }

    public float getPerimeter()

    { perimeter = 2* Math.PI *radius;

        return perimeter; } }

    public class circle

    { public static void main(String []s)

    { AreaOfCircle area=new AreaOfCircle();

        area.readRadius();

        double area1= area.getArea();

        double area2= area.getPerimeter().

        System.out.println("Area of circle:" + area1);

        System.out.println("Perimeter of circle:" + area2); } }
```

source: <https://www.javatpoint.com/java-tutorial>

ADDING A METHOD THAT TAKES PARAMETERS AND RETURN VALUE

```
int square()  
{  
    return 10 * 10;  
}
```

```
int square(int i)  
{  
    return i * i;  
}
```

```
class Box {  
    double width;  
    double height;  
    double depth;  
    // compute and return volume  
    double volume() {  
        return width * height * depth;  
    }  
    // sets dimensions of box  
    void setDim(double w, double h, double d) {  
        width = w;  
        height = h;  
        depth = d;  
    }  
}  
  
class BoxDemo5 {  
    public static void main(String args[]) {  
        Box mybox1 = new Box();  
        Box mybox2 = new Box();  
        double vol;  
        // initialize each box  
        mybox1.setDim(10, 20, 15);  
        mybox2.setDim(3, 6, 9);  
        // get volume of first box  
        vol = mybox1.volume();  
        System.out.println("Volume is " + vol);  
        // get volume of second box  
        vol = mybox2.volume();  
        System.out.println("Volume is " + vol);  
    }  
}
```

```

class Box {
double width;
double height;
double depth;

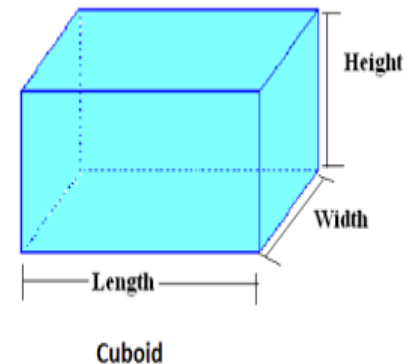
// compute and return volume

double volume() {
return width * height * depth;
}

void setDim(double w, double h, double d) {
width = w;
height = h;
depth = d;
}
}

class BoxDemo5 {
public static void main(String args[]) {
Box mybox1 = new Box();
Box mybox2 = new Box();
double vol;
// initialize each box
mybox1.setDim(10, 20, 15);
mybox2.setDim(3, 6, 9);
// get volume of first box
vol = mybox1.volume();
System.out.println("Volume is " + vol);
// get volume of second box
vol = mybox2.volume();
System.out.println("Volume is " + vol);
}
}

```



CONSTRUCTOR IN JAVA

- In Java, constructor is a block of codes similar to method. It is called when an instance of object is created and memory is allocated for the object.
- It is a special type of method which **is used to initialize the object**.
- When is a constructor called Everytime an object is created using new() keyword, atleast one constructor is called. It is called a default constructor.
- **Note:** It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class. It is because java compiler creates a default constructor if your class doesn't have any.

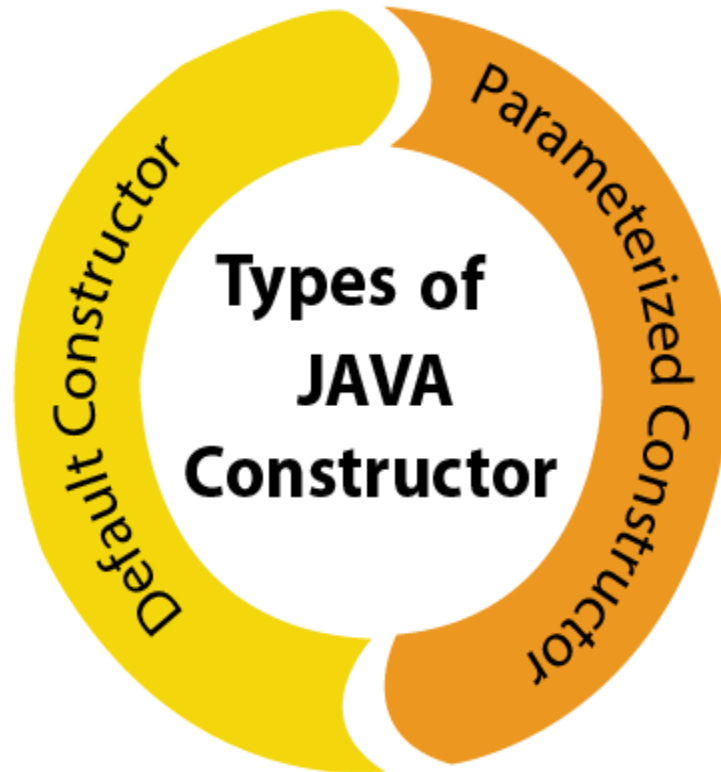
Rules for creating java constructor There are basically two rules defined for the constructor.

- Constructor **name must be same as its class name**
- Constructor must **have no explicit return type**

TYPES OF JAVA CONSTRUCTORS

There are two types of constructors in java:

- Default constructor (no-arg constructor)
- Parameterized constructor



JAVA DEFAULT CONSTRUCTOR

A constructor is called "**Default Constructor**" when it doesn't have any parameter.

Syntax of default constructor:

```
<class_name>()
```

```
{}
```

Example of default constructor

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked at the time of object creation.

```
class Bike{  
  
    Bike(){System.out.println("Bike is created");}  
  
    public static void main(String args[]){  
  
        Bike b=new Bike();  
  
    }  
}
```

Rule: If there is no constructor in a class, compiler automatically creates a default constructor.

What is the purpose of default constructor?

- Default constructor is used to provide the default values to the object like 0, null etc. depending on the type.

EXAMPLE OF DEFAULT CONSTRUCTOR THAT DISPLAYS THE DEFAULT VALUES

```
class Student3{  
  
    int id;  
  
    String name;  
  
    void display(){System.out.println(id+" "+name);}  
  
    public static void main(String args[]){  
  
        Student3 s1=new Student3();  
        Student3 s2=new Student3();  
  
        s1.display();  
        s2.display();  
  
    } }  

```

Id
Name

Id
name

Output:

0 null

0 null

JAVA PARAMETERIZED CONSTRUCTOR

- A constructor which has a specific number of parameters is called parameterized constructor.

Why use parameterized constructor?

- Parameterized constructor is used to provide different values to the distinct objects.

EXAMPLE OF PARAMETERIZED CONSTRUCTOR

```
class Student{  
    int id;  
    String name;  
    Student(int i,String n){  
        id = i;  
        name = n;  
    }  
    void display()  
{System.out.println(id+" "+name);} }  
Class Test{  
    public static void main(String args[]){  
        Student s1 = new Student(111,"Karan");  
        Student s2 = new Student(222,"Aryan");  
        s1.display();  
        s2.display();  
    }  
} }
```

source: <https://www.javatpoint.com/java-tutorial>

```
class Box

{ double width; double height; double depth;}

Box(double wid, double hei, double dep)

{   System.out.println("Constructing Box");

width = wid; height = hei; depth = dep; }

double volume()

{ return width * height * depth; } }

class BoxDemo

{ public static void main(String args[])

{ Box mybox1 = new Box(100, 200, 150);

Box mybox2 = new Box(30, 60, 90);

double vol;

vol = mybox1.volume();

System.out.println("Volume of the first box is " + vol);

vol = mybox2.volume();

System.out.println("Volume of the second box is " + vol);} }
```

CONSTRUCTOR OVERLOADING IN JAVA

- In Java, a constructor is just like a method but without return type. It can also be overloaded like Java methods.
- Constructor overloading in Java is a technique of having more than one constructor with different parameter lists. They are arranged in a way that each constructor performs a different task. They are differentiated by the compiler by the number of parameters in the list and their types.

EXAMPLE OF CONSTRUCTOR OVERLOADING

```
class Student5{  
    int id;  
    String name;  
    int age;  
    Student5(int i,String n){  
        id = i;  
        name = n; }  
    Student5(int i, String n, int a){  
        id = i;  
        name = n;  
        age=a; }  
    void display(){System.out.println(id+" "+name+" "+age);} }  
Class Test{  
    public static void main(String args[]){  
        Student5 s1 = new Student5(111,"Karan");  
        Student5 s2 = new Student5(222,"Aryan",25);  
        s1.display();  
        s2.display(); } } } source: https://www.javatpoint.com/java-tutorial
```

JAVA COPY CONSTRUCTOR

- There is no copy constructor in java. But, we can copy the values of one object to another like copy constructor in C++.

There are many ways to copy the values of one object into another in java. They are:

- By constructor
- By assigning the values of one object into another
- By clone() method of Object class

COPY THE VALUES OF ONE OBJECT INTO ANOTHER USING JAVA CONSTRUCTOR.

```
class Student6{

    int id;

    String name;

    Student6(int i,String n){

        id = i;

        name = n;  }

    Student6(Student6 s){

        id = s.id;

        name =s.name;  }

    void display(){System.out.println(id+" "+name);}

class Test{

    public static void main(String args[]){

        Student6 s1 = new Student6(111,"Karan");

        Student6 s2 = new Student6(s1);

        s1.display();

        s2.display();

    } }
```

source: <https://www.javatpoint.com/java-tutorial>

Method Overloading



Method Overriding



COPYING VALUES WITHOUT CONSTRUCTOR- COPY THE VALUES OF ONE OBJECT INTO ANOTHER BY ASSIGNING THE OBJECTS VALUES TO ANOTHER OBJECT. IN THIS CASE, THERE IS NO NEED TO CREATE THE CONSTRUCTOR.

```
class Student7{

    int id;

    String name;

    Student7(int i,String n){

        id = i;

        name = n; }

    Student7(){

    }

    void display(){System.out.println(id+" "+name);}

class test{

    public static void main(String args[]){

        Student7 s1 = new Student7(111,"Karan");

        Student7 s2 = new Student7();

        s2.id=s1.id;

        s2.name=s1.name;

        s1.display();

        s2.display();

    }

}
```

source: <https://www.javatpoint.com/java-tutorial>

THIS KEYWORD IN JAVA

In java, this is a **reference variable** that refers to the current object.

Usage of java this keyword

Here is given the 6 usage of java this keyword.

- this can be used to refer current class instance variable.
- this can be used to invoke current class method (implicitly)
- this() can be used to invoke current class constructor.
- this can be passed as an argument in the method call.
- this can be passed as argument in the constructor call.
- this can be used to return the current class instance from the method.

1) THIS: TO REFER CURRENT CLASS INSTANCE VARIABLE
THE THIS KEYWORD CAN BE USED TO REFER CURRENT
CLASS INSTANCE VARIABLE. IF THERE IS AMBIGUITY
BETWEEN THE INSTANCE VARIABLES AND PARAMETERS,
THIS KEYWORD RESOLVES THE PROBLEM OF AMBIGUITY.

```
class Student{  
  
    int rollno;  
  
    String name;  
  
    float fee;  
  
    Student(int rollno,String name,float fee){  
  
        this.rollno=rollno;  
  
        this.name=name;  
  
        this.fee=fee;  
  
    }  
  
    void display(){System.out.println(rollno+" "+name+" "+fee);}  
  
}  
  
    class TestThis2{  
  
    public static void main(String args[]){  
  
        Student s1=new Student(111,"ankit",5000f);  
  
        Student s2=new Student(112,"sumit",6000f);  
  
        s1.display();  
  
        s2.display();  
  
    }  
}
```

source: <https://www.javatpoint.com/java-tutorial>

2) THIS: TO INVOKE CURRENT CLASS METHOD

```
class A{  
  
  void m(){System.out.println("hello m");}  
  
  void n(){  
  
    System.out.println("hello n");  
  
    // m();//same as this.m()  
  
    this.m();  
  
  }  
  
}  
  
class TestThis4{  
  
  public static void main(String args[]){  
  
    A a=new A();  
  
    a.n();  
  
  }  
}
```

source: <https://www.javatpoint.com/java-tutorial>

3) THIS() :TO INVOKE CURRENT CLASS CONSTRUCTOR

- The this() constructor call can be used to invoke the current class constructor. It is used to reuse the constructor. In other words, it is used for constructor chaining.
- **Calling default constructor from parameterized constructor:**

```
class A{  
  
A( ){System.out.println("hello a");}  
  
A(int x){  
  
    this();  
  
    System.out.println(x);  
  
}  
  
}  
  
class TestThis5{  
  
    public static void main(String args[]){  
  
        A a=new A(10);  
  
    }  
}
```

CALLING PARAMETERIZED CONSTRUCTOR FROM DEFAULT CONSTRUCTOR:

```
class A{  
  
    A(){  
  
        this(5);  
  
        System.out.println("hello a");  
  
    }  
  
    A(int x){  
  
        System.out.println(x);  
  
    }  
  
}  
  
class TestThis6{  
  
    public static void main(String args[]){  
  
        A a=new A();  
  
    }  
}
```


JAVA GARBAGE COLLECTION

- In java, garbage means unreferenced objects.
- Garbage Collection is process of reclaiming the runtime unused memory automatically. In other words, it is a way to destroy the unused objects.
- To do so, we were using free() function in C language and delete() in C++. But, in java it is performed automatically. So, java provides better memory management.

Advantage of Garbage Collection

- It makes java **memory efficient** because garbage collector removes the unreferenced objects from heap memory.
- It is **automatically done** by the garbage collector(a part of JVM) so we don't need to make extra efforts.

FINALIZE() METHOD

The finalize() method is invoked each time before the object is garbage collected.

This method can be used to perform cleanup processing.

This method is defined in Object class as:

```
protected void finalize(){} 
```

gc() method

- The gc() method is used to invoke the garbage collector to perform cleanup processing. The gc() is found in System and Runtime classes.

```
public static void gc(){} 
```

SIMPLE EXAMPLE OF GARBAGE COLLECTION IN JAVA

```
public class TestGarbageI{  
  
    public void finalize(){System.out.println("object is garbage collected");}  
  
    public static void main(String args[]){  
  
        TestGarbageI s1=new TestGarbageI ();  
  
        TestGarbageI s2=new TestGarbageI ();  
  
        s1=null;  
  
        s2=null;  
  
        System.gc();  
  
    }  
  
}
```

Output:

object is garbage collected

object is garbage collected

JAVA PROGRAM GENERATES RANDOM NUMBERS

```
import java.util.Scanner;
import java.util.Random;

class A{
    int maxRange;

    void numbers()
    {
        Scanner SC = new Scanner(System.in);

        Random rand = new Random();

        System.out.print("Please enter maximum range: ");

        maxRange=SC.nextInt();

        for(int loop=1; loop<=10; loop++)
        {
            System.out.println(rand.nextInt(maxRange)); }}}

public class MyClass {

    public static void main(String args[]) {

        A a=new A();

        a.numbers(); }}
```

source: <https://www.javatpoint.com/java-tutorial>

OBJECTIVES

I. What is true about constructor?

- a) It can contain return type
- b) It can take any number of parameters
- c) It can have any non access modifiers
- d) Constructor cannot throw exception

B

2. What would be behaviour if constructor has a return type?

- a) Compilation error
- b) Runtime error
- c) Compilation and runs successfully
- d) Only String return type is allowed

- Answer: a

Explanation: The constructor cannot have a return type. It should create and return new object. Hence it would give compilation error.

3. Which keyword is used by method to refer to the object that invoked it?

- a) import
- b) catch
- c) abstract
- d) this

- Answer: d

Explanation: this keyword can be used inside any method to refer to the current object. this is always a reference to the object on which the method was invoked.

4. Which operator is used by Java run time implementations to free the memory of an object when it is no longer needed?
- a) delete
 - b) free
 - c) new
 - d) none of the mentioned

- Answer: d

Explanation: Java handles deallocation of memory automatically, we do not need to explicitly delete an element. Garbage collection only occurs during execution of the program. When no references to the object exist, that object is assumed to be no longer needed, and the memory occupied by the object can be reclaimed.

What is the output of this program?

```
1.class area
2.{
3.int width;
4.int length;
5.int area;
6.void area(int width, int length)
7.{
•this.width = width;
1.this.length = length;
2.}
3.
4.}
5.class Output
6.{
7.public static void main(String args[])
•{
1.area obj = new area();
2.obj.area(5 , 6);
3.System.out.println(obj.length + " " + obj.width);
4.}
5.}
a)                                0
b)                                5
c)                                6
d) 5 5
```

0
6
5



source: <https://www.javatpoint.com/java-tutorial>